

长春大学

本科毕业设计（论文）

面向可验证代码修改的编程智能体 ProofCode 的设计与实现

学 院 _____

专 业 计算机科学与技术

学生姓名 _____

学 号 _____

指导教师 _____

毕业届别 _____

年 月

摘要

编程智能体能够根据自然语言需求读取仓库、修改代码并执行测试，但持续任务中的版本混淆、重复工具日志和外部数据写入会增加结果核验与恢复难度。针对这些问题，本文设计并实现面向可验证代码修改的 ProofCode 系统核心后端。系统采用 Go 执行引擎与 Java Spring Boot 控制平面，使用 PostgreSQL 保存任务及批准事实，以事务 Outbox 和 ActiveMQ Artemis 实现任务派发，并在独立 Git 工作树中进行代码修改和验证。

上下文模块采用词法、符号、文本依赖线索和测试四路检索，通过倒数排名融合组织原文证据，并以项目、工作区、任务尝试、Git 提交及文件哈希绑定证据版本。压缩模块保留完整原始会话，只在模型输入副本中替换完全重复的成功纯日志。数据网关通过模式快照、受限查询中间表示、参数化 SQL、计划摘要和用户批准约束数据库操作，并提供 PostgreSQL 事务回滚和 Redis 条件补偿。系统同时实现六组固定输入的执行配置及结果比较接口。

验证采用软件测试、服务全链路夹具和合成机制检查。六组夹具均完成真实补丁与统一测试；真实 PostgreSQL 工作流的批准前修改次数为 0，批准后为 1，重复和拒绝未增加修改。重复成功日志在合成条件下可缩短模型视图，而全链路夹具未出现去重收益。由于模型与决策接口使用固定响应，本文结果证明工程机制及执行框架的可复现性，不代表真实模型的任务能力提升。后续需通过真实任务集进一步评价检索、压缩与决策路由的效果。

关键词：编程智能体；版本感知检索；上下文压缩；数据库审批；可复现验证

Abstract

Coding agents can inspect repositories, edit code, and execute tests in response to natural language requests. However, inconsistent code versions, repeated tool logs, and external data mutations make long running tasks difficult to verify and recover. This thesis presents the core backend of ProofCode, a system for verifiable code modification. A Go execution engine works with a Java Spring Boot control plane. PostgreSQL stores task and approval records, a transactional outbox and ActiveMQ Artemis dispatch tasks, and isolated Git worktrees contain code changes and verification results.

The context module combines lexical, symbol, textual dependency, and test retrieval through reciprocal rank fusion. Source evidence is bound to project, workspace, task attempt, Git revision, and file hashes. The compression module preserves the original transcript and replaces only identical successful plain logs in the model input view. A project scoped data gateway uses schema snapshots, a restricted query intermediate representation, parameterized SQL, plan digests, and explicit user approval. PostgreSQL transactions and separately approved Redis compensation handle supported failure cases. Six executable configurations share fixed inputs and a result comparison interface.

Verification includes software tests, a service level fixture, and synthetic mechanism checks. All six fixture configurations applied real patches and passed a common test command. In a real PostgreSQL approval workflow, the mutation count was zero before approval and one afterwards; replay and denial caused no additional mutation. Repeated successful logs reduced the estimated model view under synthetic conditions, while the service fixture showed no deduplication benefit. Since model and decision responses were fixed, the results establish reproducibility of the engineering mechanisms rather than gains in model task performance. Further evaluation requires real tasks and labeled retrieval data.

Keywords: coding agent; version aware retrieval; context compression; database approval; reproducible verification

目录

摘要	I
Abstract	II
第 1 章 绪论	1
1.1 研究背景	1
1.2 研究意义	1
1.3 国内外研究与产品现状	2
1.4 研究内容与方法	3
1.5 主要贡献与研究边界	4
1.6 论文组织结构	4
第 2 章 需求分析与相关技术	5
2.1 系统目标与使用场景	5
2.2 功能需求	5
2.3 非功能需求	6
2.4 需求范围与优先级	7
2.5 关键技术分析	7
2.6 本章小结	8
第 3 章 系统总体设计	9
3.1 总体架构	9
3.2 作用域模型	10
3.3 任务生命周期与可靠调度	11
3.4 Agent 执行与协作设计	12
3.5 混合代码检索设计	13
3.6 数据库与缓存网关设计	13
3.7 信任边界与安全设计	14
3.8 可观测性与任务胶囊	14
3.9 本章小结	15
第 4 章 面向工作树的多路代码检索与证据组织	16
4.1 检索需求与总体设计	16
4.2 原文分块与四路候选检索	17
4.3 排名融合与上下文预算	18
4.4 版本快照与证据一致性	18
4.5 缓存机制与受控验证	19
第 5 章 面向工具交互的上下文压缩与预算控制	20

5.1 长任务上下文与双层记录	20
5.2 完全重复成功日志的识别	21
5.3 完整工具消息组的预算控制	22
5.4 估算口径与真实模型用量	22
5.5 受控机制结果与适用范围	23
第 6 章 项目隔离的数据网关设计与实现	24
6.1 数据操作需求与设计边界	24
6.2 结构快照与受限操作表示	24
6.3 批准与一次性执行机制	26
6.4 PostgreSQL 事务与 Redis 补偿	28
6.5 审计、证据与实现局限	29
第 7 章 六组对照框架与系统验证	30
7.1 验证问题与证据层次	30
7.2 六组配置的执行契约	30
7.3 测试环境与复现方法	32
7.4 软件测试结果	33
7.5 六组全链路结果	34
7.6 数据库批准与重复执行验证	35
7.7 正式研究的补充设计	35
第 8 章 总结与展望	37
8.1 工作总结	37
8.2 局限与后续工作	37
致谢	39
参考文献	40
附录 A 复现入口与证据索引	42
A.1 源码与命令	42
A.2 证据文件	42
附录 B 参考项目与借鉴范围	43
B.1 编程智能体项目	43
B.2 数据库与上下文项目	43
附录 C 指标观测与研究边界	44
C.1 Agent 与检索指标	44
C.2 数据库指标	44

第 1 章 绪论

1.1 研究背景

近年来，大语言模型从文本生成工具逐渐发展为能够调用外部工具、读取工程文件并执行多轮任务的软件智能体。**ReAct** 将推理过程与行动过程结合，使模型可以在“思考—调用工具—观察结果”的循环中完成比单轮问答更复杂的任务[1]。在软件开发场景中，智能体可以读取源代码、搜索符号、修改文件、运行测试并根据失败信息继续修复，因此其任务对象从一段代码扩展为具有依赖关系、版本历史和构建约束的完整工程。

编程智能体需要理解需求、定位文件、执行修改、验证结果并处理执行中断。任何环节出错，都可能造成错误修改、未经批准的命令执行或过期上下文被复用。模型输出是不可信输入，文件系统、数据库和命令执行器需要独立的权限与状态控制。

代码智能体还面临上下文规模与新鲜度的矛盾。项目代码和工具输出不断累积，全部加入提示词会增加成本并干扰证据利用。**RAG** 通过外部非参数记忆提供可更新的知识来源[2]；**RepoCoder** 将反复检索用于仓库级代码生成[3]。代码片段是否可用还取决于路径、符号、测试关系和 **Git** 版本，因此不能只依据通用文本相似度。

应用代码还可能关联 **PostgreSQL**、**MySQL** 或 **Redis**。错误表名、无条件更新和危险 **DDL** 使模型直接执行任意 **SQL** 具有明显风险。数据操作需要结构化中间表示、模式快照、执行计划、批准和审计，并明确操作所属项目、任务和执行尝试。

本课题据此设计并实现 **ProofCode**，一个面向软件工程任务、支持本地工作树的可验证编程智能体后端。系统围绕任务胶囊保存用户需求、上下文证据、工具调用、审批、代码变更、验证结果和恢复信息；以项目、工作区、对话和任务建立作用域；在隔离 **Git** 工作树中执行修改；对数据库与缓存操作实施显式批准。研究重点放在可验证执行链、版本感知代码检索和受控数据操作的组合实现，不涉及基础模型训练。

1.2 研究意义

1.2.1 工程实践意义

对个人开发者而言，系统应说明修改依据、隔离原始工作区，并在失败后保留中间状态。**ProofCode** 通过事件和制品关联检索、修改与验证，使用户能够先检查补丁和测试证据，再将变更应用到干净的本地检出目录。

对团队服务而言，持久化任务、可靠派发、租约和 **attempt** 可以将模型循环与 **Web** 请求解耦，区分新旧执行，并为多个 **Runner**、审批等待和重试提供协调基础。

对数据安全而言，网关将模型操作转化为受约束计划，绑定项目、资源和模式快照，要求批准摘要与执行摘要一致，并保存执行证据。PostgreSQL 事务和 Redis 条件补偿为受控失败处理提供基础。

1.2.2 学术与应用意义

现有研究通常分别讨论检索增强生成、软件工程智能体、上下文压缩或数据库问答。本文将这些问题放在同一个可运行的执行链中，研究版本作用域、工具审计和人工审批如何影响智能体的可靠性。系统使用确定性检索、RRF 排名融合[4]和保守的日志去重，避免把未经验证的摘要当成事实；同时记录检索快照、证据范围和压缩估算，为后续实验提供可复现的观测点。

该课题采用 Go、Java 和 React 分工，涉及并发、事务、消息、Git、权限与测试，可展示从需求到受控实验的完整软件工程过程。评价依据是可解释证据和可重复对照，而非单纯增加模型规模。

1.3 国内外研究与产品现状

1.3.1 软件工程智能体

ReAct 为工具型智能体提供了推理与行动交替的基本范式[1]。SWE-agent 将这一范式用于 GitHub 软件问题，通过专门的计算机接口帮助模型浏览仓库、编辑代码和运行测试[9]；SWE-bench 则把真实软件仓库中的 Issue 与测试作为评测任务，为补丁是否真正解决问题提供了比文本相似度更严格的标准[8]。mini-swe-agent 展示了较小的“请求—执行—观察”循环[28]，有利于理解编程智能体的最小运行时。

面向实际产品的项目在交互和工程层面提供了不同参考。Aider 重视仓库地图、Git 集成和代码编辑格式[19]；Pi 将模型、消息、工具、事件、会话和上下文管理拆分为相对清晰的模块[29]；Crush 展示了 Go 语言中的会话、权限、Shell 和 MCP 组织方式[30]；Cline 强调可见的工具调用和批准交互[31]。OpenCode 提供了多模型、桌面端、SDK 与插件的产品化参考[18]，OpenHands 则更强调容器化工作区和平台执行[20]。Codex 的公开代码可作为安全执行、工作树和协议分层的参考[16]；Claude Code 是用户体验基线之一，但其公开仓库不能被表述为完整产品的自由开源实现[17]。

这些项目说明，编程智能体还需要权限、会话、版本和评测系统。ProofCode 的借鉴范围集中在执行循环、模块划分、版本控制和审阅工作流，参考项目的性能结果不属于本系统实验结果。

1.3.2 检索增强与上下文管理

RAG 通过外部检索补充模型参数记忆，能够提供来源和更新机制[2]。RepoCoder 关注仓库级代码检索与迭代生成[3]，说明代码任务的上下文应根据当前修改循环动态更新。RRF 是一种将多个排序结果进行融合的方法，在不同检索器的评分不可直接比较时仍可通过名次获得稳定组合[4]。另一方面，长上下文并不意味着模型能够均匀利用所有内容，Lost in the Middle 研究提醒系统关注证据在长输入中的位置和干扰问题[5]。LLMLingua 与 LLMLingua-2 代表模型式上下文压缩路线[6][7]，但压缩可能改变代码、SQL 或错误信息，必须单独验证。

本文使用词项、声明符号、文本依赖和测试线索的四路排序，经 RRF 合并，并为原文绑定版本与位置。压缩只替换完全重复的成功纯日志。数据库迁移预留了 pgvector 扩展、向量字段与索引，但 Runner 尚未接入 embedding 检索、AST 或历史向量库；压缩规模采用估算口径，不能据此直接计算计费节省。

1.3.3 数据库可视化与治理

数据库产品提供了可借鉴的交互形式。DBeaver 的连接树、数据网格和执行计划适合数据库运维场景[24]；Bytebase 展示了数据库变更审批、迁移和审计的工程流程[21]；ChartDB、drawDB 和 React Flow 分别提供关系图展示、拖拽建模和节点画布的参考[22][23][25]。这些系统的共同点是把数据库操作呈现为可检查的对象，而不是把所有行为隐藏在一段自然语言中。

ProofCode 当前实现项目隔离的模式、计划、审批、执行和审计。ER 图、拖拽查询和完整客户端为后续工作。

1.4 研究内容与方法

本文的主要研究内容如下：

（1）设计编程智能体的任务运行时。实现模型请求、工具注册、工具审批、取消、预算、事件记录和验证状态，并以单写者规则约束多角色协作。

（2）设计项目作用域和可靠调度机制。建立项目—工作区—对话—任务层次，使用事务 Outbox 写入任务与待发送消息，Runner 通过租约和 attempt 处理派发、续租、取消、重试和恢复。

（3）设计版本感知的混合代码检索。使用 Git HEAD、工作树 manifest、路径和文件内容哈希形成快照；对 lexical、symbol、dependency、test 四类结果进行 RRF 融合，并在每轮请求前重新验证工作树。

(4) 设计可恢复的上下文视图。原始消息用于持久化和恢复，模型视图只在满足严格条件时去重成功日志；超过预算时按完整工具调用组裁剪，避免留下不匹配的工具协议。

(5) 设计项目级数据库和缓存数据网关。资源通过服务端密钥引用访问，计划绑定 schema snapshot 和 digest，数据库写操作需用户批准，审计记录执行前后状态，失败时使用事务回滚或显式补偿。

(6) 执行 A-F 六组对照配置。固定 commit、输入和验收命令，独立任务生成比较 JSON/CSV。现有运行使用固定 LLM/Jev 响应，验证了配置生效、真实补丁、测试和导出；第七章按工程验证报告，真实模型与检索质量仍待实验。

研究采用文献分析、参考项目比较、原型实现、软件测试和受控合成验证相结合的方法。检索率、真实任务成功率和延迟收益需要后续使用带标签的任务集与实际模型响应测量，当前未由模型自述或模拟夹具推断这些结论。

1.5 主要贡献与研究边界

本文的工程贡献可以概括为三点。第一，将任务作用域、工作树版本和检索证据绑定，降低旧版本代码或其他项目内容混入模型上下文的风险。第二，在完整 transcript 可恢复的前提下提供确定性日志去重与工具组预算裁剪，形成可审计的模型上下文视图。第三，将结构化数据计划、审批、一次性授权、事务执行、审计和补偿连接到同一个任务生命周期，使数据库与缓存操作具有可检查的责任链。

本文不声称提出全球唯一的 RAG 算法，不声称已经证明低幻觉或速度提升，也不声称达到商业企业级可用性。当前后端未实现 embedding/AST 长期多版本索引；MCP 客户端和浏览器控制库尚未接入 Runner；Wails 桌面端仍依赖控制面和 Runner，不能单独代表完全离线执行；可视化数据库和缓存客户端、团队级 RBAC、强隔离的每任务容器以及真实 A-F 模型实验仍属于后续工作。论文中的实现结论依据代码与测试证据，研究效果保留为待验证问题。

1.6 论文组织结构

本文正文共八章。第一章为绪论，说明研究背景、相关工作与边界。第二章为需求分析与相关技术。第三章为系统总体设计，介绍架构、作用域与任务生命周期。第四章为面向工作树的多路代码检索与证据组织。第五章为面向工具交互的上下文压缩与预算控制。第六章为项目隔离的数据网关设计与实现。第七章为六组对照框架与系统验证，报告固定模型响应夹具及机制检查，并提出正式模型实验设计。第八章为总结与展望。

第 2 章 需求分析与相关技术

2.1 系统目标与使用场景

ProofCode 面向处理真实仓库的个人开发者和小型团队。用户创建项目、工作区和对话，提出修复、重构、测试或数据检查任务；系统准备上下文，调用模型与获准工具，返回差异、验证和审计证据。用户审查后决定是否应用制品。

Web 场景通过控制面派发任务，由 Go Runner 在隔离工作树执行；Wails 桌面壳承载共享界面，当前仍依赖服务端。SCRATCH 支持临时工作区类型，但桌面自动创建会话工程目录尚未完成。

2.2 功能需求

2.2.1 项目、工作区与对话管理

项目可以拥有多个工作区，对话属于工作区，任务引用合法的三层组合。服务端必须验证归属，拒绝跨项目标识符；未传作用域的旧客户端归入稳定的 Legacy 工作区和对话。

2.2.2 任务创建、执行与恢复

任务保存提示、模型、作用域、幂等键和执行参数，经历排队、运行、审批、验证和终结。失败或取消可增加 attempt 后重试，审批决定后可恢复；旧尝试和旧 Runner 不得改变当前状态。

2.2.3 代码工具与变更验证

Agent 提供读取、列举、搜索、补丁、命令和差异六个核心工具，执行前检查模式、路径和策略。修改位于 attempt 工作树，写入和命令默认需审批。checkpoint/recovery 供用户审查；应用制品时必须验证干净检出和 base SHA。

2.2.4 检索与上下文管理

检索应返回当前工作树中带作用域、位置和哈希的证据，融合词项、声明、依赖与测试线索。上下文管理保留原始消息，消除重复成功日志，并按完整工具组裁剪超预算输入。

2.2.5 数据库与缓存操作

数据资源按项目隔离，凭据留在服务端。Runner 请求模式、计划、EXPLAIN 和执行，用户审查计划与审计。PostgreSQL 使用参数化有界操作，Redis 使用命名空间和有界扫描。所有写操作需批准，并拒绝摘要变更、重复执行和旧 attempt。

2.2.6 实验与证据导出

系统应支持 A-F 六组固定 profile：纯 LLM、确定性安全策略、Jev 路由、混合代码 RAG、RAG 加上下文压缩和完整 ProofCode。实验服务固定 prompt、模型、temperature、源 commit、最大步骤、测试命令、重复次数和作用域，生成独立任务和 outbox 记录。比较接口应输出每组 planned、terminal、valid、succeeded、failed 和 unknown 数量，以及带样本数和缺失数的指标；缺少实际观测值时必须保留 unknown。

2.3 非功能需求

2.3.1 可靠性与一致性

任务记录和 Outbox 消息必须在同一个数据库事务中提交，避免任务已经持久化但消息没有可派发记录。Runner 使用租约领取任务并定期续租，失去租约后不能继续写入当前尝试。事件需要按 attempt 和序号幂等处理；旧 attempt 的终结事件不能覆盖新 attempt。进程在工具执行中断时，系统应把结果标为未知并保留工作树，不能盲目重放可能已经产生副作用的调用。

2.3.2 安全性与最小权限

模型输出、仓库内容和外部扩展均视为不可信输入。内置工具必须限制路径和命令；数据库资源的凭据不能返回给模型或前端；数据库和缓存写入必须经人工批准。控制面区分用户令牌和 Runner 令牌，内部端点不能由普通用户令牌调用。安全策略是执行边界的一部分，Jev 只能在固定候选工具中选择，不能生成参数、批准写入或越过确定性策略。

2.3.3 可追溯性与可复现性

事件应记录任务、attempt、工具、审批、快照、制品和终结状态。检索证据应可回到源文件的字节范围；数据库操作应能由计划摘要和审计记录关联。实验使用完整 commit 而不是可变分支，导出 JSON/CSV 保存 profile 与实际应用状态。日志和指标不得包含数据库密码、连接串或敏感行数据。

2.3.4 性能与资源边界

系统限制文件、仓库、结果、输出与上下文规模。当前以 UTF-8 字节除以四估算输入，输出上限 16,384，总预算 200,000，不能代替真实 tokenizer。检索缓存省分块和评分，仍需读取文件检查版本。

2.3.5 可维护性与扩展性

Agent、Runner、控制平面、数据适配器和界面通过明确协议交互。PostgreSQL 保存事实与租约，控制面 Redis 支持取消标记，Artemis 派发任务，HTTP/WebSocket 传递实时事件。项目 Redis 网关和进程内检索缓存各自独立。模块可单独测试，后续向量检索、可视化和多用户权限应保持任务与审批契约。

2.4 需求范围与优先级

本阶段优先交付核心后端和可复现的工程验证。功能范围、当前状态和边界如下：

范围	当前处理方式	状态
Agent 循环和工具	Go Engine、路径/策略检查、事件与预算	已实现并有 Go 测试入口
任务调度	Java 控制面、Outbox、Artemis、Runner 租约和 attempt	已实现并有集成夹具
代码上下文	四路确定性检索、RRF、快照证据、重复日志去重	已实现；无 embedding/AST
数据操作	PostgreSQL/Redis 受限网关、审批、审计、回滚/补偿	核心后端已实现；范围有意受限
客户端特色	ER 图、拖拽 Query Builder、缓存可视化、目录式对话	后续客户端工作

团队 RBAC、任意 SQL/DDl、跨存储事务、历史向量库、强任务沙箱、完整插件市场和真实模型质量结论不属于已完成范围。MCP 与浏览器库尚未接入 Runner。

2.5 关键技术分析

2.5.1 Agent 循环与工具协议

系统采用模型请求与工具执行交替的 ReAct 思路[1]。模型负责理解需求、生成代码和工具参数，执行器负责验证工具名称、参数结构、路径、权限和审批。工具返回结果作为下一轮观察输入，并同时写入事件。将工具选择和工具参数生成分开，可以让固定安全策略不依赖模型是否“自觉”遵守提示词。

2.5.2 代码检索与排名融合

RAG 的外部证据访问机制[2]适合解决代码任务中的仓库范围问题；RepoCoder 的仓库级迭代思想[3]适合说明为什么每轮修改后都应重新检索。系统使用 RRF[4]融合不同检索路径，避免要求它们共享同一种原始评分。Lost in the Middle[5]说明上下文打包需要控制数量和位置；当前实现通过证据字节预算和工具组裁剪限制输入，而不是简单追加全部历史。

2.5.3 上下文压缩

LLMLingua 与 LLMLingua-2[6][7]代表需要模型参与的压缩方法，具有更强的压缩潜力，但也需要重新验证代码、SQL 和错误栈的可用性。本系统先采用无生成式的重复日志消除，保留一份完整副本和来源哈希；对源代码、失败日志、JSON、SQL 和未知格式不作替换。该取舍牺牲部分压缩比例，换取原文可追溯性。

2.5.4 数据库事务与执行计划

PostgreSQL 的事务机制为同一连接中的失败回滚提供基础[10]；EXPLAIN 可用于在执行前查看查询计划和估计代价[11]。Redis 的 WATCH 与事务命令提供乐观并发检查[12]，缓存补偿仍需单独处理恢复条件。系统因此使用结构化 IR、模式快照、精确影响行数和审计记录，约束数据操作的范围。

2.5.5 版本控制、消息与安全

Git worktree 可以在同一仓库中创建彼此隔离的工作目录[13]，适合把每次 attempt 的修改与用户基础工作区分开。Outbox 模式将业务记录和待发布消息放在同一事务中，降低消息丢失风险[14]。系统参考 OWASP 提示注入防护建议[15]，将外部内容视为数据，并在工具执行前校验操作范围。

2.5.6 框架选型

Go 用于并发、子进程与 Git 执行；Spring Boot 提供 Web、事务、JPA、调度和安全配置，用于控制平面[26]。PostgreSQL 保存事实和租约，Redis 支持取消标记与项目缓存，Artemis 通过 AMQP 派发任务[27]。React 承载共享界面，Wails 桌面壳依赖服务端执行链。多语言拆分增加接口维护成本，也使组件职责更清楚。

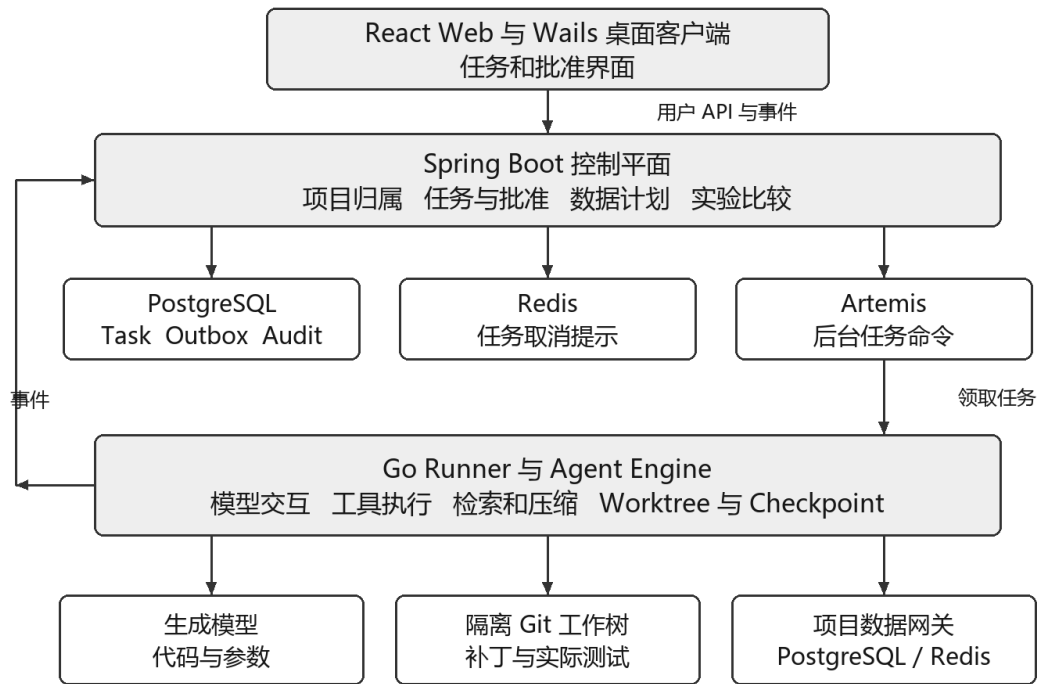
2.6 本章小结

本章明确了项目、任务、检索和数据操作中的版本、权限、批准与验证要求。下一章介绍系统架构与流程。

第 3 章 系统总体设计

3.1 总体架构

ProofCode 由共享界面、Spring Boot 控制面、Go Agent Engine/Runner 和基础设施组成。控制面将任务与 Outbox 记录在 PostgreSQL 同一事务中，再经 Artemis 派发。Runner 领取任务，在独立工作树执行模型、检索、工具和测试，回传事件。控制面 Redis 支持取消标记，项目缓存资源通过命名空间数据网关访问，持久事实和任务租约保存在 PostgreSQL。



桌面任务仍依赖控制平面和 Runner；数据修改复用用户批准

图 3-1 ProofCode 总体架构

交互层包含 React 和 Wails；控制层管理身份、作用域、任务、审批与实验；执行层组织模型循环、工具和工作树；数据层保存状态和审计；外部依赖包括模型、仓库、测试环境和用户数据资源。各层通过 DTO、事件及认证连接，模型不能直接访问数据库。

3.1.1 控制平面

控制平面使用 Spring Boot 组织 REST API、JPA 实体、事务服务、认证过滤器、Outbox Dispatcher 和 WebSocket 广播。项目和任务是长期业务实体；TaskEvent、TaskApproval、TaskArtifact、ExperimentRun 和 DataAudit 保存过程证据。用户端使用用户令牌，/internal 端点使用独立 Runner 令牌；错误派发路径不会把内部权限放行规则变成对外 API 的访问权限。

3.1.2 Agent Engine 与 Runner

Agent Engine 包含循环、Provider、工具、预算、检索、压缩和工作树等模块。Runner 已接入这些路径及测试、事件；浏览器库和 MCP 客户端尚未接入任务工具列表，不能称为完整在线能力。

Runner 通过 Artemis 消费任务消息，先向控制面 claim 当前 attempt，再周期性 renew 租约。任务处理期间定期检查取消状态，工具和子进程使用上下文取消。Runner 将任务事件按序发送到内部端点；控制面校验 attempt、租约和事件类型，写入数据库并在提交后广播给客户端。租约过期后，符合条件的任务允许由后续 Runner 显式 claim；过期本身不会由控制面定时器自动接管或重新入队，旧执行的事件不能覆盖新执行。

3.1.3 数据与消息基础设施

PostgreSQL 保存业务事实、作用域、版本和任务的 runnerId、leaseUntil。pgvector 扩展、字段与索引仅为预留，当前 Runner 使用确定性文本检索。控制面 Redis 支持取消标记，项目 Redis 支持命名空间数据操作，未实现通用协调锁、限流和 Redis Runner 心跳；检索缓存位于 Go 进程内。Artemis 派发任务与重试，逐 Token 输出通过事件或流式连接传递。

3.2 作用域模型

3.2.1 项目、工作区和对话

项目是资源归属与检索访问的逻辑作用域，工作区属于项目，对话属于工作区。任务同时保存三层标识，创建和查询必须验证完整归属。当前采用全局用户令牌和独立 Runner 令牌，尚未形成按团队用户划分的生产多租户授权体系。旧客户端未传作用域时使用 Legacy 工作区和对话。

数据资源、Redis 命名空间、检索快照和审计均绑定作用域。服务端不接受任意本机绝对路径，前端切换文件夹不能授予其他项目权限。SCRATCH 是后端类型，自动建桌面目录仍属后续功能。

3.2.2 任务与实验作用域

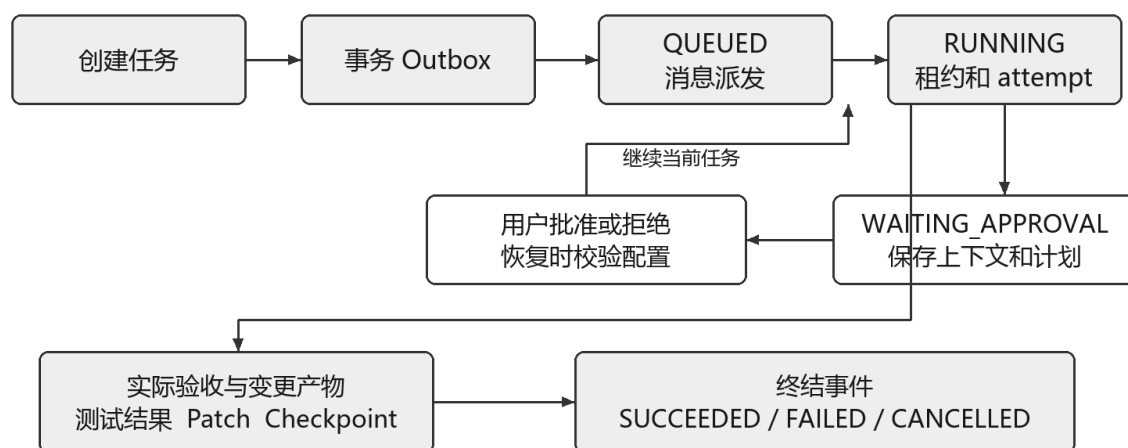
任务保存模型、提示、状态、attempt、租约和执行参数；实验另存 run、组别与 profile version。重试递增 attempt，固定实验输入保持不变，旧事件不计入当前结果。跨作用域计划和事件被拒绝。

3.2.3 检索快照作用域

一次代码检索的作用域包含索引版本、工作树绝对根、projectId、workspaceId、taskId、attemptId 和 Git HEAD。系统把候选文件按路径排序，并将路径与文件字节 SHA-256 组成 manifest，再计算 snapshotId。文件内容、证据范围和排序结果都属于该快照。每轮检索前后读取并比较工作树，发现 HEAD 或 manifest 变化就拒绝混合两个版本的结果。

3.3 任务生命周期与可靠调度

任务从 CREATED 进入 QUEUED，经 Runner 领取后进入 RUNNING；需要用户决定时进入 WAITING_APPROVAL，代码验证阶段进入 VERIFYING，最后进入 SUCCEEDED、FAILED 或 CANCELLED。FAILED 和 CANCELLED 任务可以创建新的 attempt 重新排队；WAITING_APPROVAL 任务在审批决定后恢复。成功任务不能重复重试，以避免同一制品被隐式覆盖。



中断工具结果未知时保留状态；旧 attempt 不覆盖当前终结证据

图 3-2 任务生命周期与批准恢复

3.3.1 Outbox 与派发

创建任务的事务同时写入 tasks 和 outbox_messages。Dispatcher 周期性读取未发送消息，以持久投递方式发送到 Artemis；收到确认后标记消息已发送，失败则增加尝试次数并安排后续发送。消费者消息携带 taskId 和 outboxId，Runner 通过任务 attempt 向控制面确认自己领取的是当前工作。幂等键用于防止客户端重试创建相同任务；相同键的不同输入会被拒绝。

3.3.2 租约、续租和取消

Runner claim 时提供 runnerId 和 attempt，控制面在任务锁内检查状态和版本，成功后设置租约截止时间。Runner 按固定周期 renew，控制面只接受持有当前租约的 Runner。用户取消任务后，控制面记录取消状态；Runner 轮询并取消模型请求、工具子进程和测试命令。租约和取消机制解决的是任务协调问题，不等同于对恶意程序的强安全沙箱。

3.3.3 断线恢复与未知结果

任务暂停或需要审批时，Runner 将消息、工具调用、工作树信息和 Collector 状态持久化为可恢复状态。恢复时继续使用原始 transcript，不把模型视图中的检索注入或日志引用当作新的事实来源。若进程在一个有副作用工具执行过程中崩溃，系统保留工作树并报告 outcome unknown，禁止自动重复该调用；用户可以检查 recovery 制品后决定后续操作。

3.4 Agent 执行与协作设计

3.4.1 单写者规则

系统定义 Main、Scout 和 Verifier。Main 唯一可修改文件，Scout/Verifier 使用读取、列举、搜索和 diff 的只读工具。Verifier 审查源码及测试文件，不运行命令或测试，浏览器未接入。测试由 Main 的获准命令或 Runner 统一验收执行。Scout 按提示长度和关键词启用；Verifier 需配置工具且非单 Agent。A-F 固定单 Agent。

3.4.2 工具执行链

每次工具调用先经过工具名称和参数校验，再经过工作区路径、命令 allowlist、资源作用域、审批和取消检查。允许执行后，Runner 记录 tool.requested 和 tool.started，执行器收集标准输出、错误输出、退出码和截断状态，最后写入 tool.completed 或 tool.failed。模型只看到受预算限制的观察结果；完整事件和原始消息留在任务持久化记录中。

3.4.3 上下文准备

模型请求使用原始消息的克隆作为基础。开启压缩时，系统从消息末尾向前寻找完全相同的成功纯日志，保留最近完整副本，把较早副本换为带哈希的来源引用；失败、超时、截断、源代码、JSON 和 SQL 不替换。开启 RAG 时，根据当前 task

prompt 及可选失败反馈执行四路检索，并在 48 KiB 证据预算内注入原文证据。若输入仍超过估算预算，按完整 assistant 工具调用及对应结果组从旧到新裁剪。

3.5 混合代码检索设计

3.5.1 四路检索器

Lexical 路径对查询、路径和源码进行词项拆分，识别 camelCase 和下划线标识符；symbol 路径用声明正则抽取函数、类型、接口和类名；dependency 路径从种子片段中提取声明和词法线索，查找其他文件的文本引用；test 路径优先查找路径包含 test 或 spec 的代码块。四路结果保留独立名次，便于在事件中说明证据来自哪些路径。

3.5.2 RRF 融合与证据封装

系统使用 RRF[4]合并多路名次：

$$\text{RRF}(\text{chunk}) = \sum 1 / (60 + \text{rank})$$

多路命中获得累计贡献，再按分数、路径、起始行和 ID 稳定排序。Evidence 封装作用域、快照、路径、行与字节范围、文件和块哈希、原文、分数及命中原因，模型输入因而能追溯至具体源码。

当前实现是确定性行块检索，默认每块 80 行；它不是完整 BM25、向量检索、AST 调用图或长期历史索引。工作树变化会生成新的 manifest 和 snapshotId，未修改文件可以复用解析结果，但必须重新绑定新快照；旧版本证据不会自动混入当前请求。

3.6 数据库与缓存网关设计

3.6.1 资源、模式快照与计划 IR

项目注册数据资源时只保存资源标识和服务端环境变量引用。Runner 请求 schema 时，网关连接资源并返回允许的数据库、表、字段、类型和约束，形成 schemaVersion。模型或用户提出的操作先转换成结构化 IR，计划摘要绑定 task、project、attempt、resource、provider、environment、schemaVersion 和规范化 IR。任一绑定字段变化都会使旧摘要失效。

3.6.2 PostgreSQL 执行边界

网关目前支持参数化 SELECT 以及有界 INSERT、UPDATE、DELETE。字段和表名必须来自模式快照；UPDATE 和 DELETE 必须有非空过滤器，并声明精确 expectedRows；读取最多返回 1,000 行、256 KiB，并设置十秒语句超时。执行前可以

请求 EXPLAIN，执行后写入影响行数、耗时、状态、摘要和有限错误码。自由 SQL、JOIN、函数、存储过程、DDL 和跨存储事务没有暴露。

3.6.3 Redis 执行边界与补偿

Redis 网关提供项目命名空间下的字符串 `get`、`set`、`delete`、`expire` 和有界 `cursor scan`。原始命令、脚本、KEYS、FLUSH 操作以及跨项目 key 被拒绝。涉及写入的补偿使用加密快照和 `compare-and-set` 条件，必须单独审批。外部过期、第三方改写或相同值重建都可能导致补偿冲突，因此 `COMMIT_UNKNOWN` 和 `COMPENSATION_UNKNOWN` 进入人工处理状态。

3.6.4 审批与审计

数据库写操作先产生 `data_execute` 审批事件，控制面锁定任务和计划，验证用户批准、`attempt` 和 `digest`，再在内部生成一次性 `capability` 并只保存其哈希。Runner 不能携带自己的审批凭证调用执行端点。审计台账在执行前后写入，保存计数、哈希、状态和错误码，不保存密码、连接串、查询行和缓存值。这样可以验证“批准前修改次数为零”，同时避免把敏感数据写入日志。

3.7 信任边界与安全设计

模型输出、仓库文件、工具返回、MCP 服务和数据库内容均属于不可信输入。信任边界包括浏览器到控制面、控制面到 Runner、Runner 到工作树、Runner 到数据网关以及数据网关到外部资源。每次跨边界调用都应经过身份、作用域、协议和风险控制。

安全策略分为确定性边界和模型辅助决策两层。确定性层负责路径限制、命令允许列表、资源作用域、审批、租约和取消，具有最终否决权。可选 Jev 只从预先注册的工具中选择，并把选择结果记录为事件；它不能生成文件或参数，不能批准写操作，也不能替换 SQL/路径验证。控制平面保存密钥引用，模型和客户端不获得数据库密码。

当前 Runner 是长期非 root 容器，尚无每任务临时容器、默认断网和完整资源限制，应称为受控执行器，不能宣称强沙箱。提示注入防护参考 OWASP 对外部内容与工具调用校验的建议[15]。

3.8 可观测性与任务胶囊

任务胶囊由任务输入、profile、上下文证据、模型与工具事件、审批、工作树 revision、checkpoint、测试输出、审计记录和终结结果组成。事件采用类型、任务、

attempt、序号、时间戳和有限 payload 的形式，控制面在提交后广播给客户端。检索事件记录 snapshot、HEAD、缓存命中、解析与复用数量、证据引用和是否使用失败反馈；压缩事件记录估算前后规模和原始哈希。

实验结果不从模型自述推导。只有终结事件中的 profile version、profileApplied、sourceRevision 和 experimentGroup 与计划相符时，比较器才把 run 标为 valid。缺少 provider usage、RAG 标签或数据库标注时，指标保留 unknown。现有 A–F 已通过真实服务链路执行，但 LLM 和 Jev 返回固定夹具响应；补丁、测试、数据库批准和导出证据证明执行契约成立，不能证明模型决策准确率、真实任务成功率或性能收益。该设计让系统的操作日志既服务于调试，也为第七章的对照与后续正式研究提供证据来源。

3.9 本章小结

本章给出了 ProofCode 的总体架构、作用域、任务生命周期、Agent 执行、混合检索、数据网关和信任边界。系统以 PostgreSQL 保存事实，以 Outbox 和 Artemis 保证任务可派发，以租约和 attempt 管理执行，以 Git worktree 隔离变更，以证据快照和审批台账保证可追溯。当前设计优先完成核心后端，后续客户端可在不破坏这些契约的前提下增加 ER 图、拖拽查询、缓存可视化、目录导航和多用户权限。

第 4 章 面向工作树的多路代码检索与证据组织

4.1 检索需求与总体设计

编程 Agent 需要从仓库中定位与任务有关的实现、调用线索及测试，而仓库内容会随着工具执行持续变化。若只在任务开始时建立上下文，模型后续可能根据修改前的源码继续推理；若每轮都输入全部文件，则容易超过上下文预算，并引入大量无关内容。RepoCoder 将仓库检索与代码生成结合，体现了仓库级信息对编程任务的作用[3]。本系统借鉴检索增强的基本思路，将检索对象限定为当前任务工作树中的原文，并为每段内容建立可核验的来源标识。

检索模块以任务描述为查询，按需追加最近失败工具的反馈，输出具有路径、行号、字节范围和哈希的源码片段。模块依次完成受限文件扫描、快照检查、原文分块、四路排序、排名融合与字节预算筛选。Runner 在主 Agent 的模型请求准备阶段调用该模块，检索结果只加入本轮模型视图，不改写仓库或历史工具结果。普通检索组每轮沿用原任务查询并重新校验文件，反馈组才把失败信息加入查询。因此，重新读取工作树与改变检索问题是两个分别控制的行为。

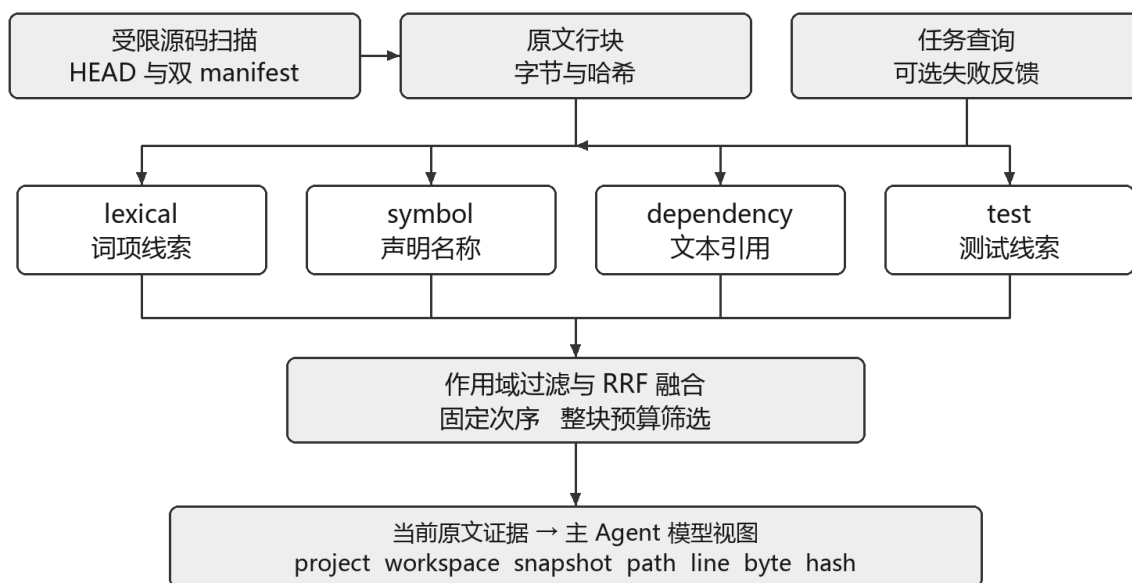


图 4-1 多路代码检索与证据组织

本设计不要求额外部署向量模型，当前实现采用词法、声明符号、文本依赖和测试四条确定性路径。这一选择使检索过程能够通过原文与规则复查，但也限制了其能力：模块没有生成 embedding，没有训练重排器，也没有建立跨语言类型系统或历史版本知识库。本章所述“依赖”仅指文本引用线索，不代表经过语法和类型解析的真实调用关系。

4.2 原文分块与四路候选检索

4.2.1 受限扫描与原文分块

扫描首先验证根目录为真实目录，并排除版本控制元数据、依赖安装目录、构建产物和已知敏感目录。文件名规则过滤环境变量、凭据及私钥类文件，内容规则检查部分秘密赋值和私钥头部；二进制或非 UTF-8 内容不进入候选集合。读取时还检查符号链接路径、打开文件的一致性及读取长度。默认上限为四千个候选文件、三十二 MiB 总内容和五百一十二 KiB 单文件；超过规定边界返回错误，不把不完整扫描伪装成完整仓库索引。秘密检测属于启发式过滤，不能替代访问权限和完整脱敏机制。

进入索引的内容保留原始 UTF-8 字节，不将换行、缩进或字符串规范化。模块通过换行字节确定行起点，默认每八十行形成一个块，末块可以不足八十行。对块记录起止行与半开字节区间，因此原文片段满足 $C=F[b_start:b_end]$ ，其中 F 为文件原始字节序列。行号方便模型和使用者阅读，字节范围支持精确复核，两者与文件哈希共同约束来源。八十行分块不保证函数完整，跨块声明和调用仍可能割裂，这是固定行块实现的局限。

4.2.2 词法与声明符号路径

词法路径从查询、路径和源码中提取字母、数字与下划线标识，统一大小写，去除部分停用词，并拆分部分驼峰和下划线名称。对命中查询词的块使用逆文档频率式权重，路径含查询词时追加权重。其评分可写为 $S_{lex}(c,q)=\sum I(t \in c)[\ln(1+(N+1)/(df(t)+1))+2I(t \in path(c))]$ ，求和范围为去重后的查询词， N 为块数， $df(t)$ 为包含该词的块数， I 为条件成立时取一的指示函数。实现按词项是否出现计分，没有完整词频建模与长度归一化，故不将其称为完整 BM25。

声明符号路径使用正则表达式提取 `func`、`def`、`class`、`interface`、`type` 等声明名称，再匹配查询中的完整名称及拆分词。完整名与局部词命中采用不同权重，使明确给出函数名的任务能够直接定位声明。正则规则跨语言使用，容易实现和复查，但不解析重载、作用域和接收者类型，也可能受到字符串、注释及特殊语法影响。因此，“符号检索”指声明名称匹配，不等价于语言服务或抽象语法树提供的语义符号解析。

4.2.3 文本依赖与测试路径

关系路径从声明符号和词法候选中选取前若干种子，当前最多取八个，提取种子的声明名、文件名词干及查询词。为减少声明重复产生的误命中，扫描引用时先移除正则识别的声明片段，再对剩余文本中的名称与文件词干进行匹配。这种方式可以发

现与种子有关的调用位置或相邻模块，却无法可靠区分同名函数、注释引用和真实调用，不能据此计算完整依赖图。

测试路径进一步识别路径中含 `test` 或 `spec` 的块，结合名称引用、查询词和文件词干匹配形成排序。它把实现附近的测试作为单独证据来源，便于模型同时检查行为约束与修改位置。文件名规则不能覆盖所有测试布局，种子误命中也会影响扩展结果。四路候选因而是可解释的互补线索，其实际召回质量仍需带文件与行范围标签的任务集评价。

4.3 排名融合与上下文预算

各检索路径的评分依据不同，直接相加会使较大数值的一路支配结果。系统采用倒数排名融合，即 $RRF[4]$ ，按候选在各路径中的名次合并。其公式为 $S_RRF(c) = \sum 1/(60 + rank_r(c))$ ，其中求和仅包含返回块 `c` 的路径，`rank` 从一开始。常数六十是当前默认配置，不是本项目通过训练得到的参数；同一块被多路检索到时累积贡献，并保存命中的路径名称。

融合前先检查 `project`、`workspace` 和 `snapshot` 归属，错误作用域的候选不参与排序。同分候选按路径、起始行及证据标识确定次序，使相同输入下的输出可重复。`Runner` 当前请求至多十二个融合结果，每路候选上限随请求数量确定。排序输出还需通过默认四十八 KiB 的证据字节预算，预算包含来源头和源码文本。某块放入后会超限时，整块省略并记录省略数量，不在块内任意截断，以保留已声明的原文范围。

排名融合解决的是不同规则输出的组织问题，不把排名分数解释成相关概率。即使多路一致命中，也可能共同受到同名标识或测试命名的影响；没有相关性标签便无法得出检索准确率。长上下文研究表明，更多输入并不必然带来稳定的信息利用[5]。本系统因此约束候选数和字节数，但这些约束仅构成预算机制，尚不能证明最佳块大小或最佳上下文长度。

4.4 版本快照与证据一致性

4.4.1 作用域与双扫描校验

任务采用独立 `Git worktree` 承载修改，使不同任务的工作目录能够分离[13]。不过 `HEAD` 只能标识已提交基础版本，不能表示 `Agent` 刚写入的未提交内容。检索模块将索引版本、绝对根路径、`project`、`workspace`、`task`、`attempt` 和 `HEAD` 共同构成 `scope`，再将排序后的文件相对路径与各文件 `SHA-256` 组成 `manifest`。快照标识定义为 `snapshot=SHA256(scope||NUL||manifest)`，字段之间使用明确分隔，避免连接歧义。

每次检索先读取 `HEAD`，随后进行两次完整受限扫描，比较两次 `manifest`，并再次读取 `HEAD`。若文件清单或 `HEAD` 不一致，本次检索返回变化错误，等待工作树稳

定后重试，而不把两种版本拼入同一份证据。双扫描不是文件系统事务锁，最终检查之后仍可能发生外部写入；它能检测两次观察之间的变化，不能保证任意并发条件下的绝对原子快照。只有通过过滤的文件进入 `manifest`，被排除内容的变化不直接改变清单。

4.4.2 证据标识与信任边界

每段证据保存快照、项目、工作区、路径、文件哈希、片段哈希、字节区间和行区间。证据标识结合快照与块位置生成，文件修改后即使路径相同也会形成新的证据标识。`blobHash` 字段使用整个文件字节的 SHA-256，不是 Git blob 对象编号；`sourceHash` 表示片段原文哈希。这样的区分使“来自哪个工作树状态”与“具体是哪段字节”能够分别复核。

检索输出可能包含源码注释和文档中的指令性文字。依据提示注入防护的基本原则[15]，`Runner` 在证据前明确说明其为来源数据，不是工具授权，并要求修改前读取当前文件。哈希能够发现内容变化，不能判断内容是否善意，也不证明代码正确。即使来源合法，证据仍不能替代命令策略、路径校验和人工审批。

4.5 缓存机制与受控验证

文件解析缓存以作用域、路径、文件字节哈希和分块行数为键。内容未变的文件复用分块，修改文件重新分块，删除文件不再进入当前缓存。查询缓存另绑定 `snapshot`、查询、反馈、返回数量、字节预算和分块参数；仅当这些条件相同才复用结果。返回时复制证据数组，避免调用者改变缓存内容。缓存位于单个检索器实例内，由互斥锁保护，不是 Redis 服务，也不是跨 `Runner` 共享索引。任务恢复或进程重启后可以重新构建。

查询缓存命中之前仍进行双扫描与 `HEAD` 检查，因此热缓存复用的是解析和排序结果，不意味着不读取仓库。当前资料未记录大仓库冷、热检索时延，不能把复用文件数量换算成速度提升。保存历史任务原始记录也不意味着系统支持历史版本检索：查询结果缓存只保留当前快照的有限条目，目标始终是当前工作树。

机制验证文件 `mechanism-results.json` 标明输入为受控合成数据，真实模型调用数为零。首次扫描四个文件，解析四个；相同查询的热调用命中缓存，复用四个、解析零个，证据保持相同。修改一个文件后解析一个、复用三个，并改变快照；删除后旧文件证据数量为零。改变项目或 `HEAD` 同样改变快照，错误项目与过期版本候选数量均为零，修改后证据与实际字节一致。上述结果验证了缓存与作用域规则在构造场景中的行为，不证明自然编程任务的召回率或代码生成质量。本章实现的贡献在于明确组织当前原文证据及其来源边界，后续仍需引入任务标签和独立验收评价实际效果。

第 5 章 面向工具交互的上下文压缩与预算控制

5.1 长任务上下文与双层记录

编程 Agent 的每次工具调用都会追加参数和结果，重复测试、构建检查及目录探索容易产生长日志。日志累积后，不仅请求成本可能增加，模型还需要在更多内容中重新定位任务约束。长上下文中的信息利用受到内容位置等因素影响[5]，单纯扩大窗口不能解决全部问题。LLMLingua 与 LLMLingua-2 分别研究了模型驱动的提示压缩和更高效的任务无关压缩方法[6][7]，为减少冗余输入提供了参考。本系统没有部署这些压缩模型，而采用适用范围更窄、行为可检查的重复日志替换规则。

上下文处理首先区分持久 transcript 与模型请求视图。前者记录用户输入、模型响应、工具参数和完整工具结果，用于审批恢复与执行追溯；后者是为当前模型请求制作的副本。压缩、检索注入及预算裁剪只作用于后者，完整记录仍按原样保存。这样可以在请求中省略冗余，同时避免恢复时把缩略内容当成真实执行结果。该设计保留了原始证据，但不能因此宣称模型获得了与全文完全相同的理解。

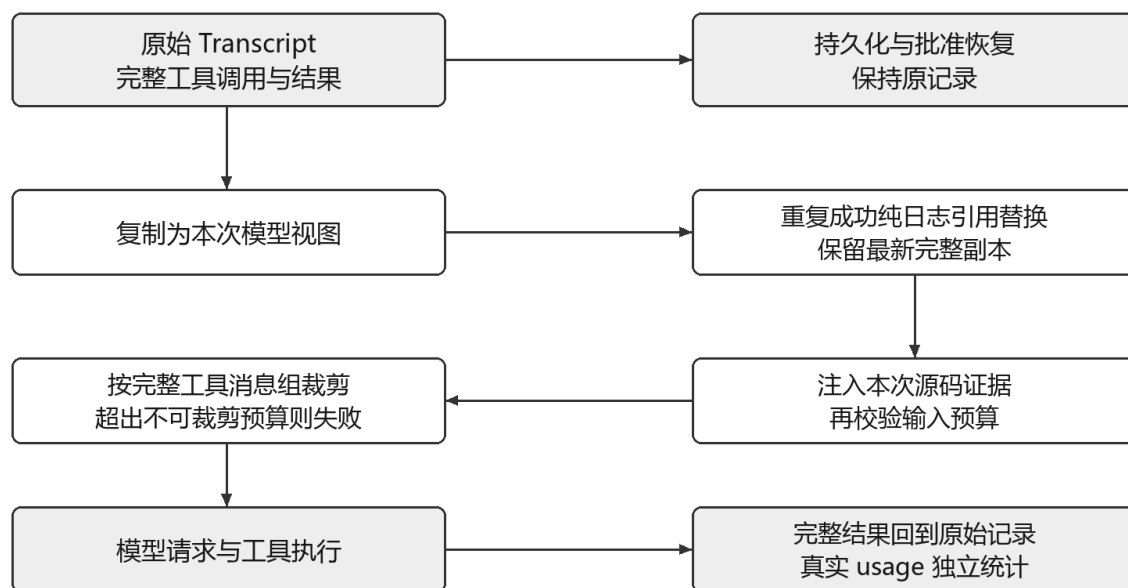


图 5-1 原始会话与模型输入视图

模型请求准备依次复制历史，按配置执行日志压缩，加入当前工作树检索证据，再依据输入预算选择完整消息组。工具执行完成后，完整结果进入原始历史而不是压缩副本。多次模型请求可以产生不同视图，但均能对应同一条持久历史；审批暂停也保存完整记录及相关计数。压缩算法不生成新的代码，不批准操作，也不决定任务是否完成。

5.2 完全重复成功日志的识别

5.2.1 保护条件

可压缩内容必须是非空工具结果，其结构能够解析，未标记为错误，并具有明确的退出状态。当前规则要求 `exitCode` 为数值零；`timedOut`、`truncated` 或 `outputTruncated` 字段若存在，必须明确为 `false`。格式未知、退出状态缺失或类型异常时保留原文，不推测执行成功。日志中出现 `failed` 或 `error:` 等失败线索同样不进入替换范围。

成功退出并不意味着所有内容都适合省略。实现进一步要求正文每个非空行属于已知的普通日志前缀，例如 `INFO`、`DEBUG`、`PASS` 及部分测试进度形式，同时拒绝合法 `JSON`、`SQL`、代码声明、可执行片段和多种结构化符号。因此，重复的文件源码、查询结果和命令参数仍保持完整。该规则可能排除不少实际成功日志，其目标是限定操作范围，而不是覆盖所有可压缩历史。未知内容保留原文会增加输入，属于可接受的保守结果。

去重判断采用完整序列化工具结果的字节哈希，而不只是肉眼相似的正文。即使两次输出语义相近，时间、元数据或换行不同也不能按同一项处理；跨工具结果只有完整内容满足相同条件时才可能进入同一去重集合。规则没有语义聚类、摘要生成和学习式价值评分，不把“类似成功”当成“完全重复成功”。这使算法行为容易复查，同时限制了自然日志中的实际命中比例。

5.2.2 倒序替换与来源引用

算法先复制消息及工具参数，计算原始历史序列化结果的 `SHA-256`，再从最新消息向前遍历。对符合条件的结果计算内容哈希，第一次遇到时保留完整文本，遇到此前的同哈希副本时生成引用说明。由于遍历方向为倒序，最新的完整输出留下，较早副本变成包含内容哈希与历史 `sourceHash` 的短文本。替换前比较字节长度，若引用不短于原文则保持原文，避免短日志因附加哈希而膨胀。

报告记录替换次数、原历史哈希及源消息引用。引用包含消息位置、工具调用标识和内容哈希，便于根据持久记录复查，但不是模型可以直接解码的原文，也没有独立按引用读取日志的接口。如果原始记录无法序列化，系统不生成无法验证的来源替换。源码中的 `Lossless` 标记表示原记录不被改写且保留完整副本，不能解释为摘要理解或模型推理已经证明无损。

这一方式与学习式提示压缩的区别体现在压缩对象和评价目标上。`LLMLingua` 系列针对提示内容的冗余开展研究[6][7]，本系统只替换有限类别中完全重复的工具结

果。它适用于反复输出同一段成功日志的场景，无法解决大量互不重复的错误信息、源码片段或长需求文档。两者不构成已经实测的性能优劣比较。

5.3 完整工具消息组的预算控制

日志去重之后，模型输入仍可能超限，因此还需执行消息组预算检查。对于普通消息，一个消息构成一组；对于包含工具调用的 `assistant` 消息，该消息和对应的全部 `tool` 结果构成同一组。系统检查调用标识非空且唯一，结果数量与标识相符，不允许孤立工具结果进入请求。预算裁剪以组为单位，避免留下调用却删去结果，或留下结果却失去其调用来源。

估算总输入包含工具定义、消息正文和调用参数。若不超限，保留全部视图；超限时先从较早历史开始移除可删除组，再处理当前用户请求后的较早组。系统消息和最近用户请求受到保护，某条用户消息及其后续交互按既定规则一起处理。这里的历史裁剪不是语义总结，被移除的组仅不进入本次请求，原始 `transcript` 仍存在。模型可能失去此前的细节，因此预算事件与删除数量需要保留，而不能宣称历史知识毫无影响。

若仅系统提示、最近用户请求和工具定义就超过预算，系统明确返回错误，不无限截断，也不把无法发送的请求当作已执行。检索证据作为系统消息加入视图，同样参与受保护内容的预算。这意味着检索模块的字节上限与模型总预算必须配合：候选内容即使在检索上限以内，也可能使请求过大。当前没有基于语义重要性的自动重排预算，裁剪规则主要保证协议一致性和可解释的失败行为。

5.4 估算口径与真实模型用量

当前上下文估算以 UTF-8 字节数为依据，对字符串 `s` 定义 $T_hat(s) = \lceil |UTF8(s)|/4 \rceil$ ，空字符串估算为零。压缩报告分别累计消息正文与序列化工具调用的估算值，得到处理前后 `T_before` 与 `T_after`。估算缩减率定义为 $R_hat = (T_before - T_after) / T_before$ 。该指标描述当前实现对输入体积的近似测量，不是精确 `tokenizer` 的输出，中文、源码、标点及不同模型的分词差异都可能改变真实 `token` 数。

模型服务返回的 `usage` 属于另一类数据，来源是具体请求的 `prompt` 与 `completion` 统计。实际费用还依赖服务计价、缓存、输出和重试等因素，不能将估算缩减率直接写成费用降低比例。压缩前后统计也不是最终请求的唯一组成：后续加入检索证据和预算裁剪可能继续改变输入。因而报告需要同时保留估算依据与真实 `usage` 的可用性，缺失服务统计时保持未知，不以估算冒充计费用量。

5.5 受控机制结果与适用范围

机制实验使用 `mechanism-results.json` 中的人工构造日志，文件明确标记为 `controlled synthetic mechanism verification`，真实模型调用数为零。实验分别重复成功日志、失败日志和源代码结果，检验去重、原记录保持及最新副本保持。单份成功日志处理前后均为一千九百五十四估算 `token`，去重次数为零，符合无重复即不替换的规则。

成功日志为两份、四份和八份时，估算值分别从三千八百九十二降至二千零二十九、从七千七百六十八降至二千一百七十九、从一万五千五百二十降至二千四百七十九，对应缩减率为 47.867%、71.949% 和 84.027%，替换副本分别为一、三和七份。全部案例的 `originalUnchanged` 与 `latestExact` 均为 `true`。结果说明完全重复且满足保护条件的长日志能够被引用替换，重复份数增加时此构造输入中的冗余比例也随之增加。

失败日志和源代码在相同重复份数下均不压缩，估算前后相同，去重次数为零。这个结果验证保护条件并非只检验成功路径，也表明重复出现本身不足以触发替换。实验没有评估模型阅读引用后的理解、任务成功率、延迟或实际费用，因此不能把最高缩减率表述为日常任务平均节省。已有全链路 E、F 组也未观察到上下文缩减，说明功能启用并不等于规则实际命中。

本章机制的可验证内容是原始记录保持、受限日志去重、来源引用和完整工具组预算。后续若评价实用价值，需要采集自然任务日志，报告实际命中比例、精确模型用量、处理开销与独立任务验收，并比较压缩后是否遗漏对下一步必要的信息。在这些数据形成之前，本系统的上下文压缩应被描述为已实现且经受控场景验证的工程规则，不作为已经证明优于现有方法的独创算法。

第 6 章 项目隔离的数据网关设计与实现

6.1 数据操作需求与设计边界

6.1.1 编程任务中的数据访问

编程智能体在修复接口、检查缓存或验证业务逻辑时，常需要同时理解代码和数据。若直接将数据库连接口令与任意 SQL 工具交给模型，模型对任务的误解就可能转化为越权查询或错误修改。数据库返回内容又会进入模型上下文，扩大敏感信息暴露范围。因此，ProofCode 将数据访问设计为控制平面中的独立网关：模型描述操作意图，网关验证范围并产生可审阅计划，用户批准后才执行具体操作。

可视化数据库产品提供了不同层面的参考。DBeaver 强调多数据源管理与查询体验[24]；ChartDB 和 drawDB 提供数据库结构展示与建模功能[22][23]；Bytebase 展示了将数据库变更置于审批流程中的方法[21]。本项目吸收这些产品关于结构可见、变更可审阅的思路，当前实现重点是后端操作契约。尚未实现拖动表节点、字段连线或可视化查询编辑器，React Flow 仅作为后续节点画布方案的参考[25]，不能据此将系统称为已经完成的可视化数据库平台。

6.1.2 归属与凭据管理

系统以 project 作为资源归属边界，workspace 表示项目内的工作空间，task 记录一次具体任务，attempt 表示同一任务的执行轮次。创建任务时校验项目、工作区与会话之间的归属关系。数据资源直接属于项目，而非单独属于工作区；数据计划通过 task 关联工作区。因此，同一项目内不同工作区可以使用同一已登记资源，不能将这种组织结构描述为工作区之间独立的数据库权限隔离。

每次数据调用均要求显式携带当前 attempt，网关拒绝旧轮次及已成功、失败或取消的任务。资源登记限定数据提供者和 dev、staging、production 环境，保存的是 secretRef。服务器在连接时解析对应环境配置，模型无需获得数据库口令。项目范围校验解决的是资源使用归属问题，当前实现也不等同于具备组织角色、细粒度成员授权和独立租户凭据管理的完整企业权限系统。

6.2 结构快照与受限操作表示

6.2.1 资源结构快照

PostgreSQL 适配器通过 JDBC 元数据读取允许 schema 中的表和字段，保存表名、字段名、类型及可空性。结构按确定的顺序组织并序列化，再计算 schemaVersion 形

成快照。计划生成必须引用已取得的快照，字段和表必须存在于该快照中。这样，模型只能针对已知结构表达操作，不能仅凭自然语言猜测任意数据库对象。

执行前再次检查实际结构与快照版本是否一致。若结构发生变化，网关将其记为前置条件失败，而不是沿用旧计划尝试修改。该检查能够发现已知元数据的变化，但不是数据库结构锁，也不承诺覆盖所有并发 DDL 竞态。快照主要面向受限单表操作，未形成包含索引、外键、视图和存储过程的完整数据库知识图谱。

资源实体与结构快照均带项目标识，检索某一版本时还要检查它属于所选资源和项目。只知道资源编号或计划编号不足以取得操作资格。对任意计划的读取和解释也保持相同校验，避免旧 Runner 通过较新计划编号推断当前轮次，再借此扩大权限。工作区的信息用于追踪操作由哪个开发任务产生，项目与执行轮次的检查则在调用过程中持续发挥约束作用。

6.2.2 Query IR 到参数化 SQL

中间表示 IR 采用结构化 JSON，将查询或修改拆分为目标对象、字段、过滤条件和操作类型。编译器只接受声明的字段，拒绝未知属性；SQL 标识符既要通过格式校验，也要属于结构快照。字段值则进入参数数组，由 PreparedStatement 绑定到占位符。标识符不能用普通参数占位符替代，所以白名单与参数绑定分别承担对象范围控制和数据值处理职责，这符合 SQL 注入防护中区分查询结构与输入数据的原则[35]。

查询只支持单表 SELECT，明确选择列，以 AND 连接有限数量的比较条件；修改只支持 INSERT、UPDATE 和 DELETE。当前不接受原始 SQL、函数、任意表达式、JOIN、DDL 和存储过程。下列 IR 表达一项预计修改一行的操作意图，仍必须经过计划生成和用户批准才能执行：

```
{
  "kind": "mutation",
  "schema": "public",
  "table": "demo",
  "operation": "update",
  "values": {"value": "approved"},
  "filters": [{"column": "id", "operator": "eq", "value": 1}],
  "expectedRows": 1
}
```

编译结果中的修改值与过滤值均使用占位符，不通过拼接字符串形成 SQL 语句。IR 限制提高了计划的可检查程度，却不能证明它符合真实业务需求。自然语言到 IR 仍由大语言模型生成；即使字段存在、SQL 可编译、影响行数符合预期，模型也可能选错对象或理解错用户意图。人工审阅与业务验收仍是必要环节。

当前比较操作限定为等于、不等于及大小比较，过滤值不得为空；字段值允许字符串、布尔值和数值等标量，拒绝任意对象和数组。结构中保存的数据库类型可辅助

模型理解，但实现没有建立覆盖全部 PostgreSQL 类型的静态类型系统，具体类型转换与约束仍由数据库执行阶段检查。由此，合法 IR 意味着通过网关的语法与范围检查，并不意味着查询一定成功或修改一定满足业务约束。

6.2.3 执行范围与计划解释

UPDATE 和 DELETE 必须带非空过滤条件，全部修改操作必须提供 `expectedRows`，取值限制在零至一千之间。实际影响行数与预期不符时回滚。它约束的是本次 DML 的影响行数，而不是业务正确性；也不保证数据库在发现不符前只处理了预期数量的记录。查询结果限制一千行、约二百五十六 KiB，语句设置十秒超时，用以避免无限结果进入工具上下文。

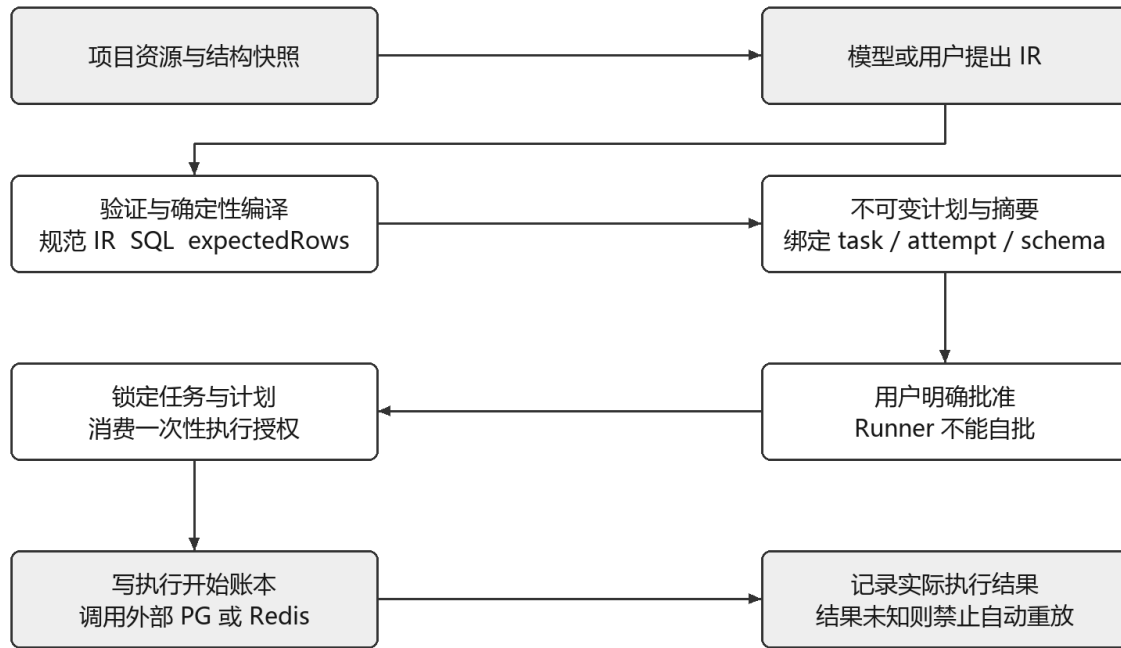
网关只为只读查询提供 EXPLAIN，使用 FORMAT JSON 返回计划，不开启 ANALYZE。PostgreSQL 文档指出 EXPLAIN 用于展示查询计划，而 ANALYZE 会实际执行语句[11]。本实现用计划信息辅助理解访问路径，不把它等同于真实运行耗时或优化效果。该接口同样核验 task、attempt 和摘要，不能因它没有修改数据就允许旧轮次绕过范围检查。

6.3 批准与一次性执行机制

6.3.1 不可变摘要与批准绑定

计划保存规范化 IR、结构版本、SQL 预览、幂等键和状态。摘要绑定任务、项目、attempt、资源、提供者、环境、schemaVersion 及规范化 IR。对象属性排序后再计算摘要，使同一结构内容得到稳定表示。批准与执行时重新计算摘要；替换资源、环境或修改值都会使原批准失效，从而避免批准的计划与执行的计划不一致。

`data_execute` 通过普通任务批准通道进入等待批准状态。用户决策时，TaskService 在事务中调用数据操作服务，同步更新任务批准记录和数据计划状态，再安排 Runner 继续执行。Runner 内部执行接口不接收由模型提交的批准决定，也不允许模型通过传入 `capability` 自行授权。允许 Runner 写文件或执行命令的配置不能替代数据计划批准，两类能力的授权边界保持独立。



生成、批准、执行开始和结束均记录审计；Redis 补偿另建批准计划

图 6-1 数据计划批准与受控执行

显式用户数据接口在批准后可以返回有效期五分钟的 `capability`。令牌使用随机字节生成，控制平面仅保存摘要，并记录有效期与使用时间；内部 `Runner` 路径则在确认持久化批准后，由控制平面产生和消费令牌。令牌不会赋予任意数据库操作权限，其效力始终依附已批准计划及当前任务轮次。过期、已消费或与计划摘要不符的请求均不能开始新的外部操作。

批准后的计划也不会立即修改目标数据库，批准和执行分别产生状态记录。用户可以拒绝一个已经编译成功的计划，而拒绝记录不会被模型的下一次调用覆盖。现有执行契约以批准状态为统一前置条件，不能认为查询计划天然绕过批准。`SQL` 预览通常显示占位符，所以审阅者还需结合 `IR` 中的参数值、过滤条件、资源环境和预期行数理解实际目标，仅看预览语句不足以完成有效审阅。

6.3.2 锁顺序、取消和重放

执行开始统一按任务、操作计划的顺序加锁。在控制平面事务内核验归属、轮次、批准与令牌，消费令牌并写入 `EXECUTION_STARTED`，然后才调用外部数据库。任务取消也取得任务锁，因此在执行账本开始之前，取消可以阻止操作取得执行资格。一旦执行已开始，取消不能撤回已发往外部提供者的事务，系统不作即时停止所有副作用的承诺。

计划幂等键限定在 `task` 和 `attempt` 范围；同一键对应相同摘要时返回既有计划，对应不同 `IR` 时拒绝。对已经确认完成的操作，在任务仍有效的前提下，重复执行只

返回摘要结果，不再次修改。任务终态则直接拒绝执行。EXECUTING 表示已启动但尚无确认结果，不能因暂时没有完成事件就再次执行。进程在开始记录后崩溃，也需要核对外部实际状态，而不是自动重放。

状态类别	处理原则
PENDING、DENIED	没有有效批准，不能执行
APPROVED	校验摘要、轮次、有效期后取得一次执行资格
EXECUTING	已开始外部操作，阻止并发重复执行
已确认完成或回滚	保留确认结果，重放不产生新修改
提交或补偿结果未知	停止自动重试，等待人工核对

6.4 PostgreSQL 事务与 Redis 补偿

6.4.1 有界 DML 与提交结果

PostgreSQL 修改在关闭自动提交的独立事务内执行，随后比较实际影响行数与 expectedRows。只有一致时才调用 commit；不一致则 rollback 并返回限定错误码。事务原子性为这种先执行、检查再提交的方式提供基础[10]。普通执行异常在尚未尝试提交且回滚得到确认时，记录已确认回滚的失败状态，而不会向模型暴露完整数据库异常。

控制平面账本事务与目标数据库事务不是同一个分布式事务。如果提交请求已经发送，但连接中断导致响应无法确认，系统记录 COMMIT_UNKNOWN；若回滚也无法确认，同样不能声称数据没有变化。自动重试可能重复已成功的修改，因此未知状态只能交由人工核对。数据库触发器若造成外部网络等副作用，也不属于本适配器可保证撤销的范围。系统未实现跨 PostgreSQL 与 Redis 的原子事务或全局回滚。

6.4.2 Redis 命名空间与恢复快照

Redis 适配器仅支持字符串 get、set、delete、expire 及游标 scan。物理键由项目标识、环境和逻辑键组合，拒绝调用者直接传入内部前缀。set 限制字符串大小并要求有限 TTL；scan 匹配项目命名空间，限制返回载荷并保留下一游标。COUNT 是扫描提示，不能认为它严格等于每页结果数量。网关不开放 KEYS、FLUSH、脚本或任意原生命令，减少跨项目访问和大范围清理的可能性。

写入前取得旧值、值摘要、TTL 与绝对过期时间，并以 AES-256-GCM 加密为私有快照。服务器要求配置三十二字节密钥，每次加密生成随机 nonce；快照不作为公开结果返回。执行阶段通过 WATCH 监视键，比较当前值与预期快照，再用 MULTI、

EXEC 提交修改。Redis 事务的乐观锁机制可检测监视期间的并发变化[12]，比较不通过则拒绝覆盖；它不是对所有历史变化的完整检测。

6.4.3 单独批准的 CAS 补偿

补偿只适用于有恢复快照且已确认提交的 Redis 操作。它生成新的计划，绑定原计划和快照摘要，再独立取得用户批准。执行时校验当前值摘要及过期状态是否仍与原操作完成后的状态相符，之后恢复旧值和剩余有效期；旧值原先不存在时删除键。若当前键已被其他业务修改或旧 TTL 已到期，补偿拒绝执行，避免恢复过时数据。

该方案恢复的是一个受限键的已记录状态，不是数据库时间旅行。即使摘要相同，若外部程序曾修改后又恢复成相同值及过期时间，快照比较也无法完整识别这种历史变化。补偿遇到连接中断则记录 `COMPENSATION_UNKNOWN`，不能将网络异常解释为一定补偿失败并重试。因此，事务回滚、条件补偿和未知结果核对是三种不同机制，分别适用于不同执行阶段。

补偿生成还要求原任务处于可操作的当前轮次，因此当前实现不能在任务终态后任意恢复历史键。它更适合任务执行过程中发现需要撤销时，由用户审阅新计划再处理。补偿计划生成和执行之间也可能发生变化，最终是否恢复仍取决于执行时的条件比较。操作人员需要结合业务含义确认恢复是否适当，不能将已存在加密快照理解为必然可以安全恢复。

6.5 审计、证据与实现局限

网关在计划生成、批准或拒绝、执行开始、快照准备和执行结束时写审计记录。持久化结果摘要只保留计数、状态、限定错误码及哈希等字段，不把读取到的数据库行和缓存明文作为计划结果保存。凭据只在服务器解析，私有恢复快照也不出现在公开 DTO 中。但用户提交的 IR 值本身需要用于审阅并存储在计划内；读取结果仍会通过工具返回进入任务上下文，故该机制不等于全链路数据脱敏。

验证入口包括契约与批准流程测试、实际参数化 DML 测试、可选的真实 PostgreSQL 和 Redis 适配器检查，以及经消息链路、Runner 与用户批准的端到端流程。当前常规报告中的真实适配器检查因环境条件未启用而跳过；端到端夹具另验证了真实 PostgreSQL 批准更新。不同证据分别支持范围拒绝、事务行为和流程连通性的判断，具体结果统一在第 7 章分析，不以某次固定夹具成功推导模型能够准确操作真实业务数据库。

本章形成了资源归属、计划审阅、批准执行及失败记录的受限数据网关。当前尚无可视化数据库界面，未支持 JOIN、DDL、Migration 和全局回滚。后续画布应输出同一 IR 并复用执行契约，业务判断仍需服务端约束与用户审阅。

第 7 章 六组对照框架与系统验证

7.1 验证问题与证据层次

编程智能体的评估需要区分软件链路能够执行与模型能够解决实际任务。前者关注任务是否按配置启动、工具是否真正修改文件、测试是否执行、数据操作是否经过批准；后者关注复杂问题的解决率、错误决策、检索质量与费用。SWE-bench 通过真实仓库问题和验收测试组织评估，SWE-agent 进一步研究执行接口对问题解决的影响 [8-9]。本文借鉴固定仓库输入和执行验收的原则，将六组配置落实为独立任务，同时保留对各类结论所需证据的区别。

本章采用三层验证。第一层是单元测试和服务内集成测试，检查解析、作用域、状态机、事务与统计规则。第二层是服务全链路夹具，它使用真实控制平面、消息中间件、Runner、Git 和 PostgreSQL，模型及 Jev 接口返回固定响应。第三层是受控合成机制检查，它调用现有检索与压缩实现，主动改变文件内容、项目标识和日志重复次数。后两层没有使用真实生成模型，因此本章结论限定为工程可复现性和机制行为，不给出模型能力提升结论。

论文对应的后端基线提交为 69b70f53aad3c117f2b79288cb82c43080eda68c。服务夹具中的任务仓库提交为 0719074f2109bdd03b81a5235c9d446b36486682，二者分别表示被验证系统的代码和系统所处理的实验输入。这个区别可以避免把程序版本与待修复仓库版本混为一谈。检索机制检查另建临时仓库，其提交由运行脚本生成，复现应比较约束和断言，不要求临时提交编号与一次存档完全相同。

7.2 六组配置的执行契约

7.2.1 对照组配置

系统定义版本化执行配置 proofcode.experiment.v1。A 至 F 不是结果页的标签，每个配置都在 Runner 创建工具注册表、决策客户端和上下文准备函数时生效。表 7-1 给出配置。六组都保留工作树隔离、路径限制、测试命令边界和数据库批准等强制约束。这些约束用于保护执行环境，不作为可以通过实验开关取消的权限。

表 7-1 六组执行配置

组别	模型侧工具	附加机制	主要比较目的
A	不开放	最终统一 diff 由独立评估器处理	纯文本模型方案的执行基线
B	开放	确定性工具安全策略	策略进入工具执行路径

组别	模型侧工具	附加机制	主要比较目的
C	开放	必需 Jev 工具路由	固定候选决策与参数生成分工
D	开放	四路代码检索	当前源码证据进入模型请求
E	开放	代码检索与日志去重	在 D 基础上考察去重机制
F	开放	策略、Jev、检索、去重与失败反馈	完整组合配置及数据工具

这些配置属于组合对照。A 与 B 同时改变工具可用性和附加策略；F 启用多个机制。因此，F 与 A 之间即使出现差异，也不能单独归因于 Jev、检索或压缩。D 与 E 更适合考察去重的影响，而进一步分析各组件贡献时还需要在后续任务集中补充单因素消融。这一限制并不妨碍验证六组确实执行，但会限制后续研究的因果解释。

C 与 F 要求 Jev 成功产生有效路由。服务缺失、响应无效、低置信度或模型调用违反已选工具约束时，任务显式失败，不能退化为其他组后继续统计成功。普通产品任务可以在可选决策服务故障时恢复常规模型选择；对照实验则必须维护配置身份。Jev 输出的概率仅是侧车的选择分数，本章不把它解释为工具正确率或安全保证。

7.2.2 固定输入与任务创建

实验请求固定用户问题、模型 ID、仓库 URL 快照、完整源码提交、最大步骤、测试命令、temperature、重复次数及项目作用域。控制平面在一个事务内写入 Experiment、各组 Run、对应 Task 和 Outbox。每组重复次数允许配置为 1 至 20；本次服务夹具使用每组一次。创建 API 支持 Idempotency-Key，同键同请求返回已有实验，同键不同请求返回 409，从源头避免网络重试建立两份实验。

源码提交必须为完整 40 位或 64 位十六进制标识。Runner 解析或获取指定 commit 后以 detached 方式检出，并从该起点建立工作树。即使远端默认分支发生变化，实验也不能改用新的分支 HEAD。无法取得指定提交时保留失败记录。审批恢复还校验执行配置指纹，防止恢复时替换问题、模型端点、策略或测试命令而沿用旧结果。

A 组没有模型侧工具，可以在最终回答中返回一个标准 Git 统一 diff。独立评估器检查路径、文件类型和补丁完整性，先运行 `git apply --check`，再应用补丁。它不是向 A 组提供的 Agent 工具。各组最终都运行相同测试命令，模型口头声称测试通过不能产生 `testsPassed=true`。当前验收命令能执行真实子进程，但正式研究仍应采用由独立评估器保管的测试集，避免智能体修改可见测试使结果失真。

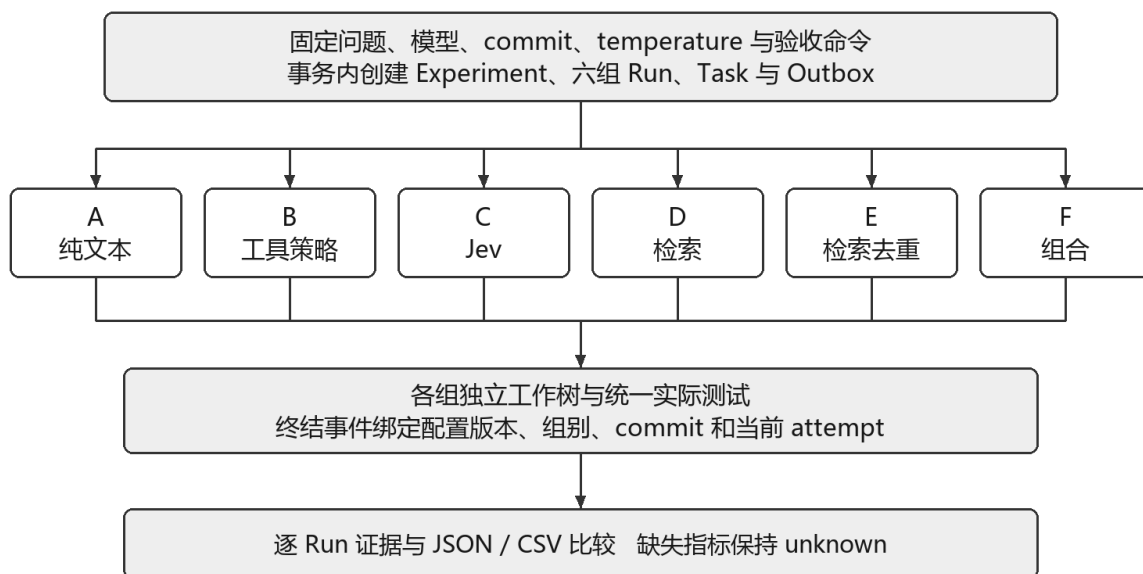


图 7-1 六组对照的执行与比较链路

7.2.3 终结证据与缺失值

每个 Run 关联一个独立 Task，终结事件携带实际配置版本、是否应用配置、源码提交、组别和观测结果。比较器只纳入当前 attempt 的终结事件；旧重试产生的事件不能覆盖新 attempt。读取任务状态与事件时可能观察到不同数据库快照，系统以已提交的当前 attempt 终结事件类型确定最终状态，以免一条已经完成的任务在一次查询中仍被统计为运行中。

有效结果要求配置和提交匹配。比较结果同时显示计划数、终结数、有效数、失败数、配置失败数和未知数；没有启动或配置失效的组仍保留在表中。数值平均值与 p95 只对有效且有观测的样本计算，并附样本数和缺失数。p95 采用 nearest-rank 口径，对排序后的 n 个观测取第 $\text{ceil}(0.95n)$ 个值。只有一个样本时该数值等于自身，不能据此描述尾延迟稳定性。

布尔指标在每个计划 Run 均有效且有该项观测时才输出完整比例，否则返回 null。模型用量同样要求全部请求均报告 usage；部分请求缺失会令总用量保持 unknown。实际供应商 Token、模型视图的字节估算和固定夹具用量是三种不同证据，不能混合计算费用。Recall@K、MRR、NDCG、正确行号和 Query IR 准确率需要标注答案，当前没有标注就不以默认零值填充。

7.3 测试环境与复现方法

后端使用 Go 1.24 系列和 Java 17，控制平面 Spring Boot 版本为 3.4.5。服务夹具由 Compose 启动 PostgreSQL 16、Redis 7.4 和 ActiveMQ Artemis 2.39.0，并使用独立网络和卷，不发布宿主端口。模型 SSE 与 Jev HTTP 响应、合成源码仓库由

smoke/experiments 提供。验证请求经公开 API 进入，随后通过 Outbox、消息派发、Runner 领取和事件回调返回结果，没有直接向比较器注入成功终结事件。

夹具运行包含构建和服务启动阶段，summary 的 elapsedSeconds 记录的是 verify 脚本自身经过时间。本次该值为 7.29 秒，不包含此前镜像构建和服务预热，不能作为部署时间。各 Run 的 durationMs 从 Runner 处理任务开始到终结，包含仓库准备、工具和验证；如果发生人工审批，还包含等待。正式速度比较应另外记录审批等待与模型请求耗时。

现有验证入口包括 Go 模块测试、Maven 测试、Compose 全链路验证和 run_mechanism_bench.ps1。复现脚本与原始结果分别保存，运行者应固定后端版本、服务镜像、输入提交及配置，再执行相应命令。重新运行的耗时、临时仓库提交和随机任务 ID 允许变化，而组数、配置身份、统一验收及批准前不执行等契约应保持一致。

7.4 软件测试结果

当前 Maven Surefire 报告包含 37 条测试，其中 35 条执行完成，失败与错误均为 0，2 条 LiveDataAdapterTest 因未启用环境条件而跳过。跳过用例不能计入执行通过数。报告覆盖实验创建与比较、任务审批和租约、HTTP 错误分发、数据网关以及 PostgreSQL 适配器。Go 全量测试及 vet 在该基线开发验证中通过，本文不把测试包数量转换为真实用户任务的成功率。

表 7-2 列出有具体断言支撑的行为。PostgreSQL 适配器的常规事务测试使用 H2 的 PostgreSQL 模式执行实际参数化 DML；它可以证明代码对受影响行数不符的回滚行为，但不能替代所有 PostgreSQL 方言及运行条件的真实验证。服务夹具另使用真实 PostgreSQL 检验批准流程，两种证据互相补充。

表 7-2 关键测试覆盖

验证入口	检查内容	证据范围
Go 实验集成测试	六组执行、固定提交、A 组 diff、必需 Jev、审批恢复	Git 和测试进程真实，模型响应固定
Go 上下文测试	作用域、版本、原文字节、完整工具消息组	检索及模型输入机制
Maven 任务与实验测试	幂等创建、旧 attempt 过滤、快照一致性、缺失值	控制平面事务与统计
Maven 数据测试	批准、摘要、令牌、重放、脱敏及未知提交	数据状态机与安全契约
H2 适配器测试	参数绑定、过滤限制、行数不符回滚	真实 JDBC 事务，非全面方言测试

验证入口	检查内容	证据范围
服务级 PostgreSQL 夹具	批准前不写、批准后一次、拒绝及重复不写	真实数据库与服务联动

HTTP 安全回归检查还验证合法内部请求保持 409 等业务状态，外部越界对象返回 404，未授权内部路由与错误分发保持 403。错误处理阶段如果绕过认证，原始路由的保护就可能失效，因此需要在实际 HTTP 服务入口验证，不能只测控制器方法。当前全局用户 Token 与 Runner Token 分离，但尚未引入团队用户角色和细粒度权限，本节测试不构成多租户安全认证。

7.5 六组全链路结果

服务夹具在同一合成 Go 仓库上修复加法函数误用减法的缺陷，六组各建立一个 Task 并通过统一测试。表 7-3 的耗时来自原始 compare.csv。模型响应固定，每组一个样本，仅用于核查执行与配置，不比较模型性能。

表 7-3 六组服务夹具实测记录

组别	有效 Run 与测试	时长 ms	检索次数及缓存命中	Jev 决策及应用
A	1 条，通过	228	0 / 0	0 / 0
B	1 条，通过	299	0 / 0	0 / 0
C	1 条，通过	325	0 / 0	4 / 4
D	1 条，通过	336	4 / 2	0 / 0
E	1 条，通过	370	4 / 2	0 / 0
F	1 条，通过	375	4 / 1	4 / 4

A 组模型请求一次，其他组请求四次；夹具每次固定返回输入 100、输出 20 的 usage。因此 A 记录 100/20，B 至 F 记录 400/80，这只是模拟供应商报告的计数，无法表达模型成本或上下文实际长度。B 与 F 各记录一次工具错误及一次高风险拦截，来自固定响应尝试读取受策略保护的文件，不表示模型在真实任务上的高危概率。C 与 F 的四次 Jev 决策同样是固定 HTTP 输出，证明路由生效，不衡量决策准确率。

E 组压缩估算累计为 1157 到 1157，F 组为 1199 到 1199，两组去重消息数均为 0。该任务没有产生符合条件的重复成功长日志，所以全链路记录没有 Token 缩减收益。D/E 命中两次查询缓存，F 命中一次；F 包含失败反馈查询变化，快照也随源码修改变化。此处缓存计数只能说明本次路径被触发，不能据此推断所有仓库的命中率或延迟收益。

本次结果说明六组固定输入、真实补丁、统一测试、执行配置与 JSON/CSV 导出相互对应。它尚未说明 F 组比 A 至 E 更能修复复杂缺陷，也未证明低幻觉、检索率提升和压缩后正确率保持。正式模型研究必须在实际任务集上重新运行这些配置。

7.6 数据库批准与重复执行验证

服务夹具在独立 PostgreSQL 中建立测试行和修改计数。任务生成单行更新计划并停在 WAITING_APPROVAL。验证器在批准前主动调用执行入口，取得 HTTP 409，数据库值仍为 original 且修改计数为 0；批准后 Runner 恢复并执行计划，值变为 approved，计数为 1。任务终结后重复执行仍为 409，拒绝另一任务的同类计划也未增加计数。

表 7-4 批准流程中的真实数据库状态

检查点	执行响应或状态	数据值	累计修改次数
用户批准前	WAITING_APPROVAL	original	0
主动尝试未批准 执行	HTTP 409	original	0
用户批准后完成	COMMITTED	approved	1
终结后再次执行	HTTP 409	approved	1
另一计划被用户 拒绝	DENIED	approved	1

验证脚本还读取审计接口，断言批准计划只有一次 EXECUTION_STARTED 和一次 EXECUTION_FINISHED，且存在 USER_APPROVED；拒绝计划存在 USER_DENIED 而没有 EXECUTION_STARTED。当前 summary 保留计数和检查结果，没有存入完整审计明细，所以本文将其作为脚本断言证据，不把它写成已经完成独立审计覆盖率研究。单次批准更新也不能替代未知提交、外部触发器和并发补偿的全部失败场景。

批准前误执行次数为 0 是该工作流的实测值，也是状态机设计要求。它并非由模型自述或前端按钮状态推导，而是由受保护执行入口的拒绝响应和独立数据库修改计数共同确认。SQL 语义是否符合用户真正意图仍需要用户审查计划；合法参数化 SQL 也可能在业务上不正确。

7.7 正式研究的补充设计

正式实验应选取具有固定起点、清楚需求和独立验收测试的任务，覆盖单文件修复、跨文件接口修改、测试补全及版本混淆情形。同一任务的 A 至 F 采用相同模型与

模型参数，各组独立工作树，顺序随机化，并记录失败和重试。重复次数、任务规模与停止条件应在运行前确定，不能只挑选成功案例。当前 API 支持每组最多 20 次重复，任务集的具体大小需结合可用资源确定，本文不虚构已开展的样本数量。

检索评估需要为查询标注正确文件、提交和行范围。Recall@K 为前 K 条证据覆盖相关标注的比例，MRR 基于首个相关证据的倒数排名，NDCG 需要相关等级及理想排序。代码行命中还应明确重叠判定和版本约束；旧版本相同行号不能算当前版本命中。缺失这些标签时，工具次数和缓存次数无法替代检索质量。

压缩研究应将 D 与 E 配对，同时记录供应商 usage、估算模型视图、去重数量、实际任务验收和错误类型。长日志任务与源码密集任务应分别观察，避免把重复日志的机制上限当作通常收益。数据库研究则应建立合法与危险 IR、批准与拒绝、行数偏差、取消、提交未知和并发改写样本，检验参数化执行、误报、审计与补偿。DDL 尚未实现，所以 Migration 回滚成功率仍为待测且前提未具备。

本章完成的是可复现执行框架及工程机制验证。真实模型的任务完成率、正式 RAG 排名指标、显存峰值、断线恢复成功率与自然语言到 IR 准确率尚无足够样本。本项目保留 unknown 并说明所需证据，为后续实验提供可用入口，而不将工程夹具包装成研究性能成果。

第 8 章 总结与展望

8.1 工作总结

本文围绕编程智能体的代码修改、上下文来源和外部数据访问，设计并实现 ProofCode 核心后端。系统以 Go 执行引擎和 Spring Boot 控制平面划分职责，使用 PostgreSQL 保存任务事实，通过事务 Outbox 和 Artemis 派发后台任务，由 Runner 在独立 Git 工作树中执行修改。Task、attempt、工具事件、批准记录及 Patch 构成可审查的任务结果，使生成式模型的建议进入具有限制和记录的实际执行过程。

代码上下文部分实现词法、符号、文本依赖线索和测试四路检索，并用 RRF 融合候选。项目、工作区、任务 attempt 和 HEAD 参与快照作用域，文件内容哈希构成 manifest，证据保存原文及行号、字节范围。修改文件、删除文件和切换提交后，后续请求重新校验快照，从工程机制上降低旧版本内容被当作当前源码的机会。当前方法采用确定性行块与启发式检索，没有实现完整语法树和历史向量索引，其有效范围在实现章节中明确给出。

上下文部分保留完整原始会话，只对模型输入副本中的完全重复成功纯日志进行引用替换，并以完整工具调用组执行输入预算管理。受控合成检查证实重复日志满足条件时模型视图缩短，错误日志和代码保持原文；全链路夹具中的 E/F 没有满足去重条件，因此未发生缩减。该结果体现了策略的条件性，尚不能作为真实供应商 Token 节省结论。

数据网关部分实现项目资源、模式快照、受限 Query IR、参数化 SQL 和不可变计划。批准、短期一次性授权及执行账本共同约束数据库修改，PostgreSQL 采用事务回滚，Redis 采用加密快照和另行批准的条件补偿。真实数据库夹具中，批准前修改次数为 0，批准后为 1，重复与拒绝均没有增加修改。结果证明该受控 workflow 可执行，业务意图正确性和更广泛故障覆盖仍需专项评估。

六组对照配置已经成为实际执行路径，同一固定提交对应独立任务、共享验收命令及逐 Run 导出。服务夹具中的六组全部完成统一测试，但模型与 Jev 响应是固定的，因此本文结论限定为配置生效、链路联动和证据一致性。本文的主要成果是一个可检查、可恢复、能支持后续对照研究的后端原型，而非已经证明超越成熟商业 Agent 的性能结论。

8.2 局限与后续工作

系统进入日常开发使用之前，还需要对真实仓库和模型开展持续验收。首先应补充带标签的任务集，严格执行六组配对比较，记录每个失败案例，再决定是否保留或调整 Jev、反馈检索和压缩策略。压缩只处理明确重复日志，源码及错误信息仍占据

较多输入；后续可研究按证据可恢复的层级上下文管理，并通过正确率与恢复成本共同评估。

检索可进一步接入语言级解析器和语义检索，但必须维持版本及项目边界。多版本历史索引应明确提交选择、保留周期和删除语义，避免相似代码跨版本混淆。增量索引需要减少每步双扫描的成本，同时证明并发修改时证据仍可核验，不能只追求缓存命中数。

数据功能的下一阶段是将可视化查询及缓存管理客户端接入既有计划接口。用户拖拽表、字段或条件时，仍应生成同一 IR 并展示受影响行数、环境、摘要和批准状态。复杂查询、更多缓存类型和 DDL 迁移需要分别定义权限、验证及失败语义；PostgreSQL 事务与 Redis 条件补偿不能扩展声称为跨存储全局回滚。

可靠性方面，需要建立团队用户与角色、细粒度权限、每任务资源限制和更强的进程隔离，并将浏览器与 MCP 扩展正式接入 Runner。针对消息中断、Runner 崩溃、未知提交与人工批准等待的故障实验，应保存原始事件和对账记录。只有这些功能和真实任务评估完成后，才能判断系统是否达到预期的日常可用性。

致谢

ProofCode 的设计与实现参考了编程智能体、数据库管理和上下文处理领域的公开研究与项目。感谢相关研究者和项目维护者公开论文、文档和代码，使本课题能够检查已有方案的实现方式，并在此基础上形成自己的系统设计。本文对所引用的研究与项目列出来源，并区分参考思路与已实现功能。

参考文献

- [1] YAO S, ZHAO J, YU D, et al. ReAct: Synergizing Reasoning and Acting in Language Models[EB/OL]. (2022-10-06)[2026-10-08]. <https://arxiv.org/abs/2210.03629>.
- [2] LEWIS P, PEREZ E, PIKTUS A, et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks[EB/OL]. (2020-05-22)[2026-10-08]. <https://arxiv.org/abs/2005.11401>.
- [3] ZHANG F, CHEN B, ZHANG Y, et al. RepoCoder: Repository-Level Code Completion Through Iterative Retrieval and Generation[C/OL]//Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing. 2023: 2471-2484[2026-10-08]. <https://aclanthology.org/2023.emnlp-main.151/>. DOI: 10.18653/v1/2023.emnlp-main.151.
- [4] CORMACK G V, CLARKE C L A, BUETTCHE S. Reciprocal rank fusion outperforms condorcet and individual rank learning methods[C/OL]//Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval. 2009: 758-759[2026-10-08]. <https://doi.org/10.1145/1571941.1572114>.
- [5] LIU N F, LIN K, HEWITT J, et al. Lost in the Middle: How Language Models Use Long Contexts[EB/OL]. (2023-07-06)[2026-10-08]. <https://arxiv.org/abs/2307.03172>.
- [6] JIANG H, WU Q, LIN C Y, et al. LLMingua: Compressing Prompts for Accelerated Inference of Large Language Models[C/OL]//Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing. 2023: 13358-13376[2026-10-08]. <https://aclanthology.org/2023.emnlp-main.825/>. DOI: 10.18653/v1/2023.emnlp-main.825.
- [7] PAN Z, WU Q, JIANG H, et al. LLMingua-2: Data Distillation for Efficient and Faithful Task-Agnostic Prompt Compression[EB/OL]. (2024-03-19)[2026-10-08]. <https://arxiv.org/abs/2403.12968>.
- [8] JIMENEZ C E, YANG J, WETTIG A, et al. SWE-bench: Can Language Models Resolve Real-World GitHub Issues?[EB/OL]. (2023-10-10)[2026-10-08]. <https://arxiv.org/abs/2310.06770>.
- [9] YANG J, JIMENEZ C E, WETTIG A, et al. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering[EB/OL]. (2024-05-06)[2026-10-08]. <https://arxiv.org/abs/2405.15793>.
- [10] PostgreSQL Global Development Group. PostgreSQL 16 Documentation: Transactions[EB/OL]. [2026-10-08]. <https://www.postgresql.org/docs/16/tutorial-transactions.html>.
- [11] PostgreSQL Global Development Group. PostgreSQL 16 Documentation: Using EXPLAIN[EB/OL]. [2026-10-08]. <https://www.postgresql.org/docs/16/using-explain.html>.
- [12] Redis. Transactions[EB/OL]. [2026-10-08]. <https://redis.io/docs/latest/develop/interact/transactions/>.
- [13] Git project. git-worktree Documentation[EB/OL]. [2026-10-08]. <https://git-scm.com/docs/git-worktree>.
- [14] RICHARDSON C. Pattern: Transactional outbox[EB/OL]. [2026-10-08]. <https://microservices.io/patterns/data/transactional-outbox.html>.

- [15] OWASP Foundation. LLM Prompt Injection Prevention Cheat Sheet[EB/OL]. [2026-10-08]. https://cheatsheetseries.owasp.org/cheatsheets/LLM_Prompt_Injection_Prevention_Cheat_Sheet.html.
- [16] OpenAI. Codex[CP/OL]. [2026-10-08]. <https://github.com/openai/codex>.
- [17] Anthropic. Claude Code[CP/OL]. [2026-10-08]. <https://github.com/anthropics/claude-code>.
- [18] anomalyco. OpenCode[CP/OL]. [2026-10-08]. <https://github.com/anomalyco/opencode>.
- [19] Aider-AI. Aider[CP/OL]. [2026-10-08]. <https://github.com/Aider-AI/aider>.
- [20] OpenHands. OpenHands[CP/OL]. [2026-10-08]. <https://github.com/OpenHands/OpenHands>.
- [21] Bytebase. Bytebase[CP/OL]. [2026-10-08]. <https://github.com/bytebase/bytebase>.
- [22] ChartDB. ChartDB[CP/OL]. [2026-10-08]. <https://github.com/chartdb/chartdb>.
- [23] drawDB. drawDB[CP/OL]. [2026-10-08]. <https://github.com/drawdb-io/drawdb>.
- [24] DBeaver. DBeaver[CP/OL]. [2026-10-08]. <https://github.com/dbeaver/dbeaver>.
- [25] xyflow. React Flow and Svelte Flow[CP/OL]. [2026-10-08]. <https://github.com/xyflow/xyflow>.
- [26] Spring. Spring Boot 3.4 Reference Documentation[EB/OL]. [2026-10-08]. <https://docs.spring.io/spring-boot/3.4/index.html>.
- [27] OASIS. OASIS Advanced Message Queuing Protocol (AMQP) Version 1.0, Part 0: Overview[EB/OL]. (2012-10-29)[2026-10-08]. <https://docs.oasis-open.org/amqp/core/v1.0/os/amqp-core-overview-v1.0-os.html>.
- [28] SWE-agent project. mini-swe-agent[CP/OL]. [2026-10-08]. <https://github.com/SWE-agent/mini-swe-agent>.
- [29] earendil-works. Pi[CP/OL]. [2026-10-08]. <https://github.com/earendil-works/pi>. 原仓库地址 <https://github.com/badlogic/pi-mono> 已重定向至该地址。
- [30] Charm. Crush[CP/OL]. [2026-10-08]. <https://github.com/charmbracelet/crush>.
- [31] Cline. Cline[CP/OL]. [2026-10-08]. <https://github.com/cline/cline>.
- [32] 长春大学教务处. 关于做好 2024 届本科生毕业设计（论文）工作的通知[EB/OL]. (2023-12-06)[2026-10-08]. <https://jwc.ccu.edu.cn/info/1033/3620.htm>.
- [33] 长春大学教务处. 附件.zip（关于做好 2024 届本科生毕业设计（论文）工作的通知附件）[EB/OL]. [2026-10-08]. https://jwc.ccu.edu.cn/system/_content/download.jsp?urltype=news.DownloadAttachUrl&owner=1453642039&wbfileid=65C361DC378581129429582586036838.
- [34] ProofCode project. ProofCode core backend source at 69b70f53aad3c117f2b79288cb82c43080eda68c[CP/OL]. [2026-10-08]. <https://github.com/lgcyyusakura/proofcode/tree/69b70f53aad3c117f2b79288cb82c43080eda68c>.
- [35] OWASP Foundation. SQL Injection Prevention Cheat Sheet[EB/OL]. [2026-10-08]. https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html.

附录 A 复现入口与证据索引

A.1 源码与命令

ProofCode 公开仓库与本章使用的固定后端提交见参考文献[34]。工程夹具、机制检查与正式模型实验采用不同入口，运行前应确认服务环境及数据用途。Compose 工程夹具只使用可丢弃测试资源；它的固定模型响应不代替真实模型研究。

```
# 仓库根目录: Go 测试
go -C agent-engine test ./...
go -C agent-engine vet ./...

# control-plane 目录: 常规 Java 测试
mvn -s settings.xml test

# 仓库根目录: 服务级工程夹具
docker compose -f compose.experiments.yml up --build -d runner
docker compose -f compose.experiments.yml run --build --rm verify

# 仓库根目录: 合成检索与去重检查
powershell -File thesis/tools/run_mechanism_bench.ps1
```

正式实验通过 POST /api/experiments 创建，传入固定模型、完整 commit、问题、测试命令和重复次数。C/F 需要有效 Jev 服务。GET /api/experiments/{id}/compare 可导出逐 Run JSON 或 CSV。生成结果后仍须按实际模型、任务来源、配置失效和缺失字段进行解释，不能仅凭接口返回成功补齐论文指标。

A.2 证据文件

本文的资料结构与 Word、PDF 归档形式参照长春大学教务处公开的 2024 届工作通知及其附件[32-33]。下面的索引将复现实验所需材料与论文叙述对应，便于核查来源和适用范围。

文件	记录内容	可以支持的结论
evidence/fixture-summary.json	服务夹具 summary 副本	六组与真实 PostgreSQL 批准流程
evidence/fixture-compare.csv	六条逐 Run 记录	观测值、缺失值和固定提交
evidence/mechanism-results.json	合成日志与检索结果	去重、原文、版本和缓存
evidence/test-report-summary.json	Surefire 统计和哈希	35 条通过、2 条跳过
evidence/provenance.json	文件哈希和后端基线	证据完整性与版本追踪

附录 B 参考项目与借鉴范围

B.1 编程智能体项目

参考项目用于了解执行循环、编辑协议、工作区和批准设计。借鉴设计思路与直接复用依赖应区分，ProofCode 没有继承这些项目的评测结果。参考库部分副本来自 2026 年 9 月 29 日的 HEAD 压缩包，缺少冻结 commit，因此本文使用项目名称和访问日期描述参考范围，不以当前主页内容推定该历史副本的所有功能。

项目	文献编号	本课题参考内容
Codex	[16]	权限批准、执行边界和任务事件
Claude Code	[17]	终端编程工作流；公开仓库不等于完整开源核心
OpenCode	[18]	会话、Provider、工具与客户端组织
Aider	[19]	RepoMap、编辑格式和 Git 变更审查
OpenHands	[20]	执行环境与任务组织，注意版本变化
mini-swe-agent	[28]	模型—动作—观察的简洁循环
Pi	[29]	消息、工具、事件与会话模块划分
Crush	[30]	Go 工程组织、会话和权限
Cline	[31]	可见工具结果和用户批准交互

部分项目使用商业或特定许可证，公开可访问不表示可以任意复制。本文没有以项目星数或营销描述作为功能优越性依据。实际 Go 依赖由 go.mod 固定，Java 依赖由 pom.xml 固定；检索中的 RRF 与上下文相关方法则按研究文献列出。

B.2 数据库与上下文项目

Bytebase[21]提供数据库变更治理参考；ChartDB[22]和 drawDB[23]提供结构画布与建模参考；DBeaver[24]提供数据源和查询界面参考；React Flow[25]是未来节点画布的候选组件。它们支持本项目后续可视化设计，不构成当前后端已经实现拖拽数据库界面的证明。LLMLingua 及其后续研究[6-7]提供压缩方法参考，当前实现未引入其模型或压缩推理代码。

附录 C 指标观测与研究边界

C.1 Agent 与检索指标

指标类别	当前可采集数据	尚需补充的研究证据
工具与策略	调用、错误、无效调用、策略拦截计数	真实任务分母及风险样本标签
任务和测试	Task 终态、实际验收命令结果	独立验收、真实任务完成定义
用量与时延	完整 usage 或 unknown、durationMs	真实模型请求、审批等待分解和足量样本
恢复与资源	检查点和 attempt 机制	断线故障实验、显存峰值采集
检索排名	原文证据、快照、排名及缓存	Recall@K、MRR、NDCG、文件与行标签
压缩影响	估算前后、去重数、原记录保持	真实 usage、配对任务验收与压缩错误

C.2 数据库指标

自然语言到 Query IR 准确率需要带目标 IR 的任务标签；IR 到 SQL 执行正确率需要独立业务验收；EXPLAIN 通过与危险操作拦截、误报率需要分别定义合法及危险样本。当前真实批准 workflow 观测到批准前修改次数为 0，但审计完整率、恢复成功率和未知提交处理质量仍需专项任务集。Migration 依赖 DDL 迁移实现，当前没有该功能，不能统计其回滚成功率。缺失字段保留 unknown，不能用零值表示未执行的研究。